
Clustering

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Review: Unsupervised Learning
- Clustering Overview
- Clustering Algorithms:
 - K-means Clustering
 - Hierarchical Clustering
 - DBSCAN

Unsupervised Learning Overview

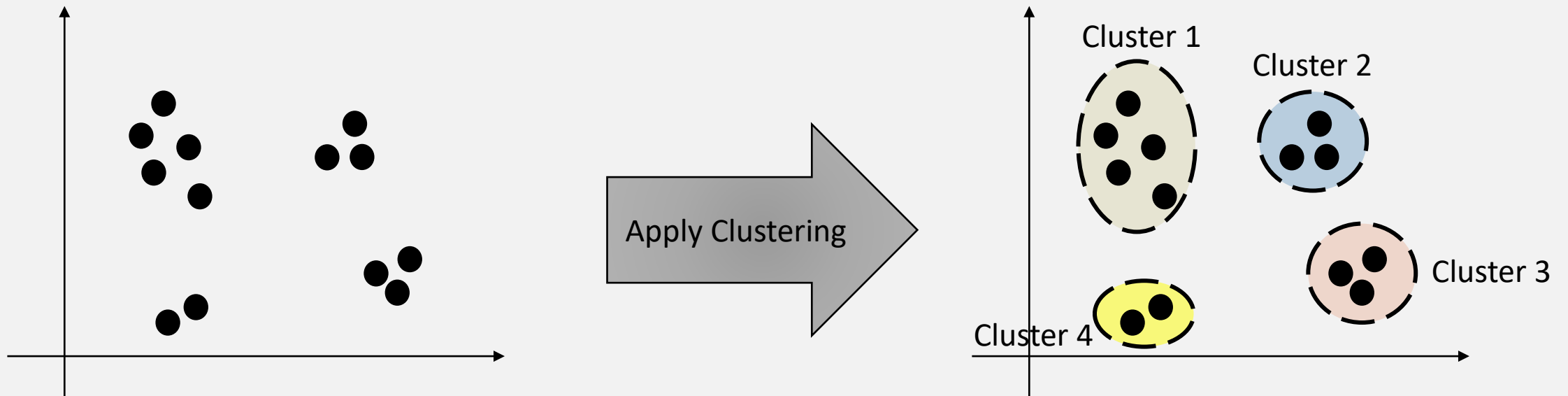
- The unsupervised learning problem consists of **unlabelled dataset** $S = \{x_1, x_2, x_3, \dots, x_m\}$.
- Note that we do not have labels y_i .
- **Goal:** Find interesting patterns, structure, or groupings in the data.
- Common unsupervised tasks:
 - Clustering
 - Dimensionality reduction.
 - Anomaly detection

Clustering (1)

- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, group the instances into groups based on their distances to each other.
- How do we measure distance between x_i and x_j ? One simple way is via vector distance. Many other ways that is a function of the features.
- How many groups do we split the data into? Can fix this to a specific number, or make it adaptive based on a performance measure.

Clustering (2)

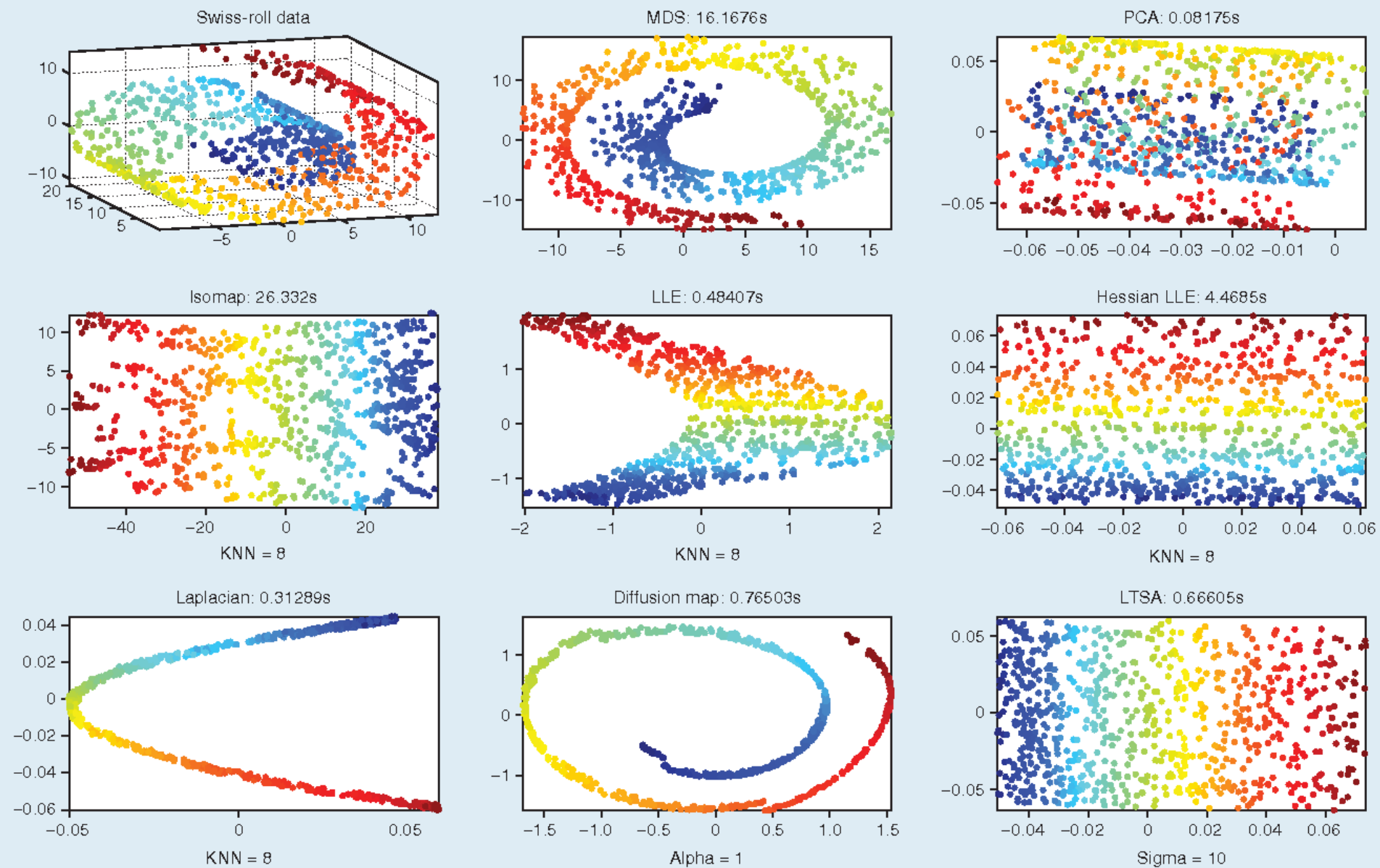
- Suppose we have a dataset with just 2 features and 13 instances. We want to cluster into 4 groups/clusters.



- Can analyze each cluster or group in more detail.
- Common application: visualization, consumer habits analysis, social network analysis.

Dimensionality Reduction (1)

- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, where x_i has d features, reduce the number of features from d to k .
- Typically, the data is mapped to lower dimensions while preserving the *structure* or *distance* of instances to each other.
- Common application is for visualization. That is map, the dataset from d dimensions to 2 or 3 dimensions and plot.
- The higher the dimensions, the harder it is to reduce to 2 or 3 dimensions while preserving the structure of the data.
- Other common application is to reduce the memory size of a high dimensional dataset. Fewer features -> smaller memory size.



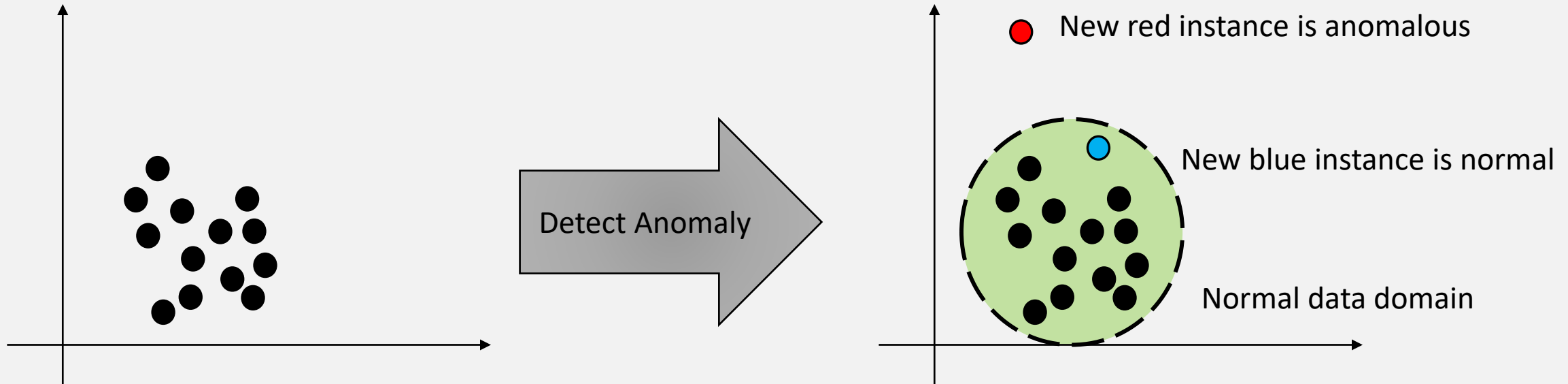
3D Swiss-roll data (top-left) is reduced to 2D using eight dimensionality reduction techniques.

Image from: <https://ieeexplore.ieee.org/document/5401149>

Anomaly detection (1)

- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, train a model to learn the **normal** pattern of an instance.
- The trained model can then be used to judge if a new instance is an outlier (anomaly, NOT normal, alarming).
- Can be used to detect alarming situations.
- Example 1: we have data on the purchasing habits of a person, we can detect if he purchases something that is out of the norm.
- Example 2: we have data from multiple sensors attached to oil pumps, we can detect if an oil pump's sensor readings are anomalous.
- Note that as we gather data over time, what is normal can shift. E.g. the purchasing habits of a person can change over time.

Anomaly Detection (2)



Clustering Overview

Clustering Overview

- **Given:** An unlabelled dataset set $S = \{x_1, x_2, x_3, \dots, x_m\}$.
- **Output:** A clustering of the instances into groups or categories based on their similarity. That is, each instance is assigned a cluster identifier like a number or string.
- Define a similarity or distance measure between any two instances x_i and x_j . Common measures to be aware of:

Euclidean (L2) distance: $L2(x_i, x_j) = \sqrt{\sum_{q=1}^n (x_{iq} - x_{jq})^2}$

Manhattan (L1) distance: $L1(x_i, x_j) = \sum_{q=1}^n |x_{iq} - x_{jq}|$

cosine similarity: $\cos(\theta) = \frac{x_i^T x_j}{\|x_i\| \|x_j\|} = \frac{\sum_{q=1}^n x_{iq} x_{jq}}{\sqrt{\sum_{q=1}^n x_{iq}^2} \sqrt{\sum_{q=1}^n x_{jq}^2}}$

Measure based on angle θ between two vectors. That is, two vectors are similar if the angle between them is smaller. Commonly used for text data where a word is represented as a vector of numbers.

- **Note** depending on the distance measure used, not-rescaling vs rescaling the features will affect the clustering results.

Method name	Parameters	Scalability	Usecase	Geometry (metric used)
K-Means	number of clusters	Very large <code>n_samples</code> , medium <code>n_clusters</code> with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters, inductive	Distances between points
Affinity propagation	damping, sample preference	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Graph distance (e.g. nearest-neighbor graph)
Mean-shift	bandwidth	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Distances between points
Spectral clustering	number of clusters	Medium <code>n_samples</code> , small <code>n_clusters</code>	Few clusters, even cluster size, non-flat geometry, transductive	Graph distance (e.g. nearest-neighbor graph)
Ward hierarchical clustering	number of clusters or distance threshold	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, transductive	Distances between points
Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, non Euclidean distances, transductive	Any pairwise distance
DBSCAN	neighborhood size	Very large <code>n_samples</code> , medium <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, outlier removal, transductive	Distances between nearest points
OPTICS	minimum cluster membership	Very large <code>n_samples</code> , large <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, variable cluster density, outlier removal, transductive	Distances between points
Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation, inductive	Mahalanobis distances to centers
BIRCH	branching factor, threshold, optional global clusterer.	Large <code>n_clusters</code> and <code>n_samples</code>	Large dataset, outlier removal, data reduction, inductive	Euclidean distance between points

K-means Clustering

K-means Overview [1]

- K-means is an iterative technique that assigns instances to K clusters.
- Consider the two-feature dataset to the right.
- K-means starts by randomly initializing the K cluster **centroids**. Cluster membership is color coded in the plot.
- Then it proceeds in iterations:
 - **Assignment**: assigns data points to nearest centroid.
 - **Update**: updates the centroids to be the mean of assigned data points.

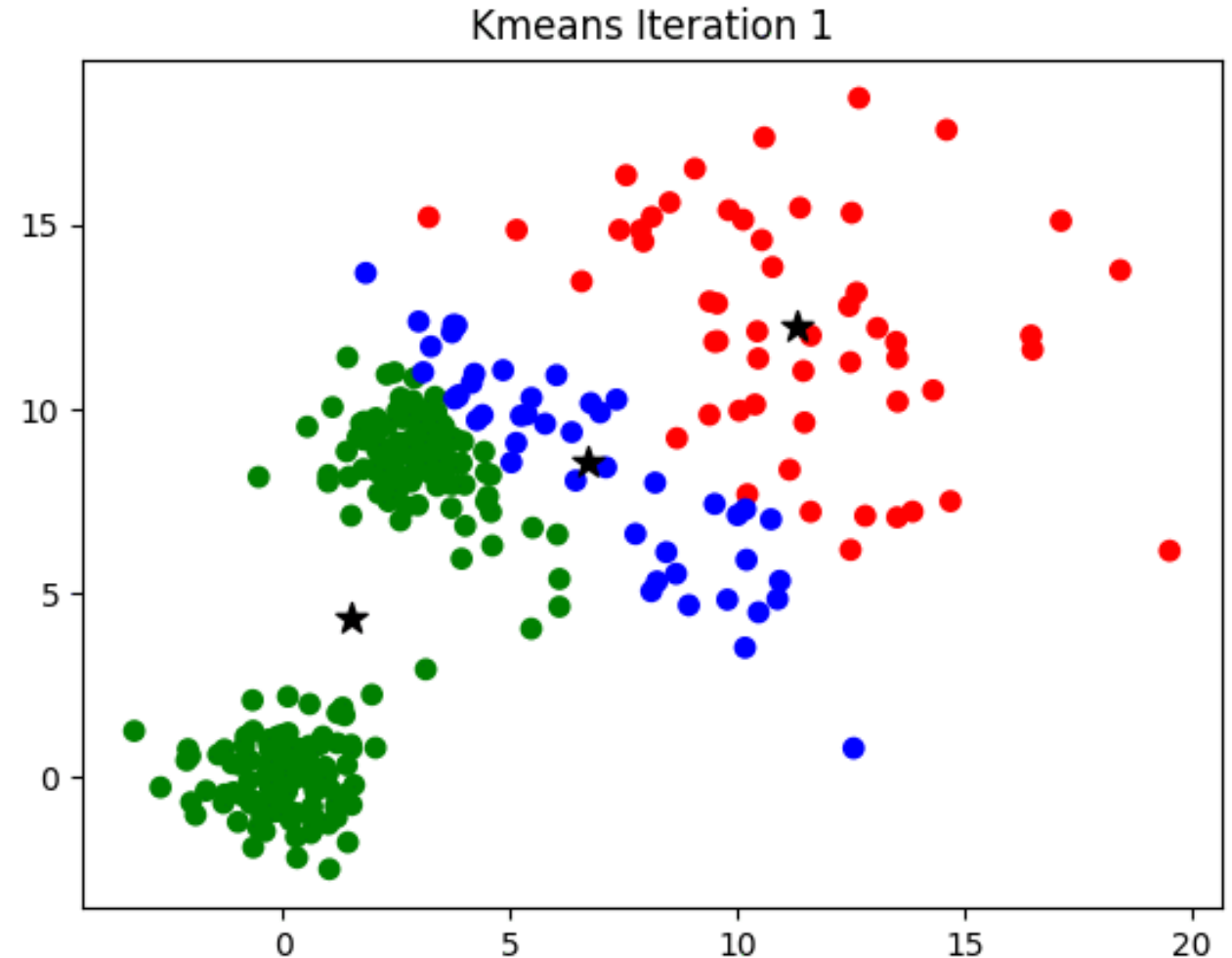
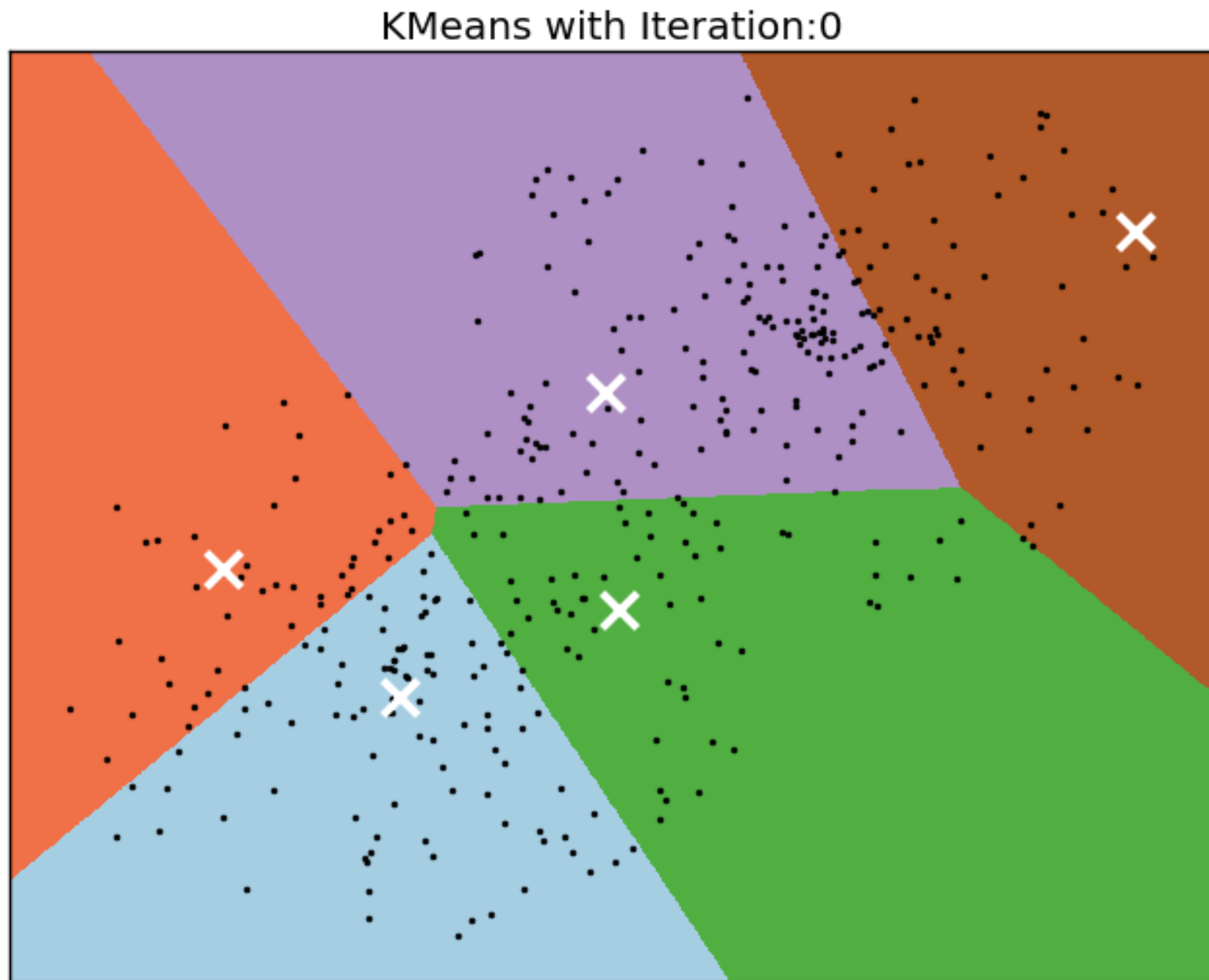


Image from: [link](#)

K-means Overview [2]

- Note that the cluster assignments correspond to Voronoi diagrams based on centroid location.
- K-means *implicitly* uses Euclidean distance. More on this later.
- See also this website for nice visualization of k-means: <https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>

Image from: [link](#)



K-means Overview [3]

- K-means minimizes the **within-cluster sum-of-squares**. It does this over iterations of Assignment and Update steps. The algorithm always converges after finite number iterations.

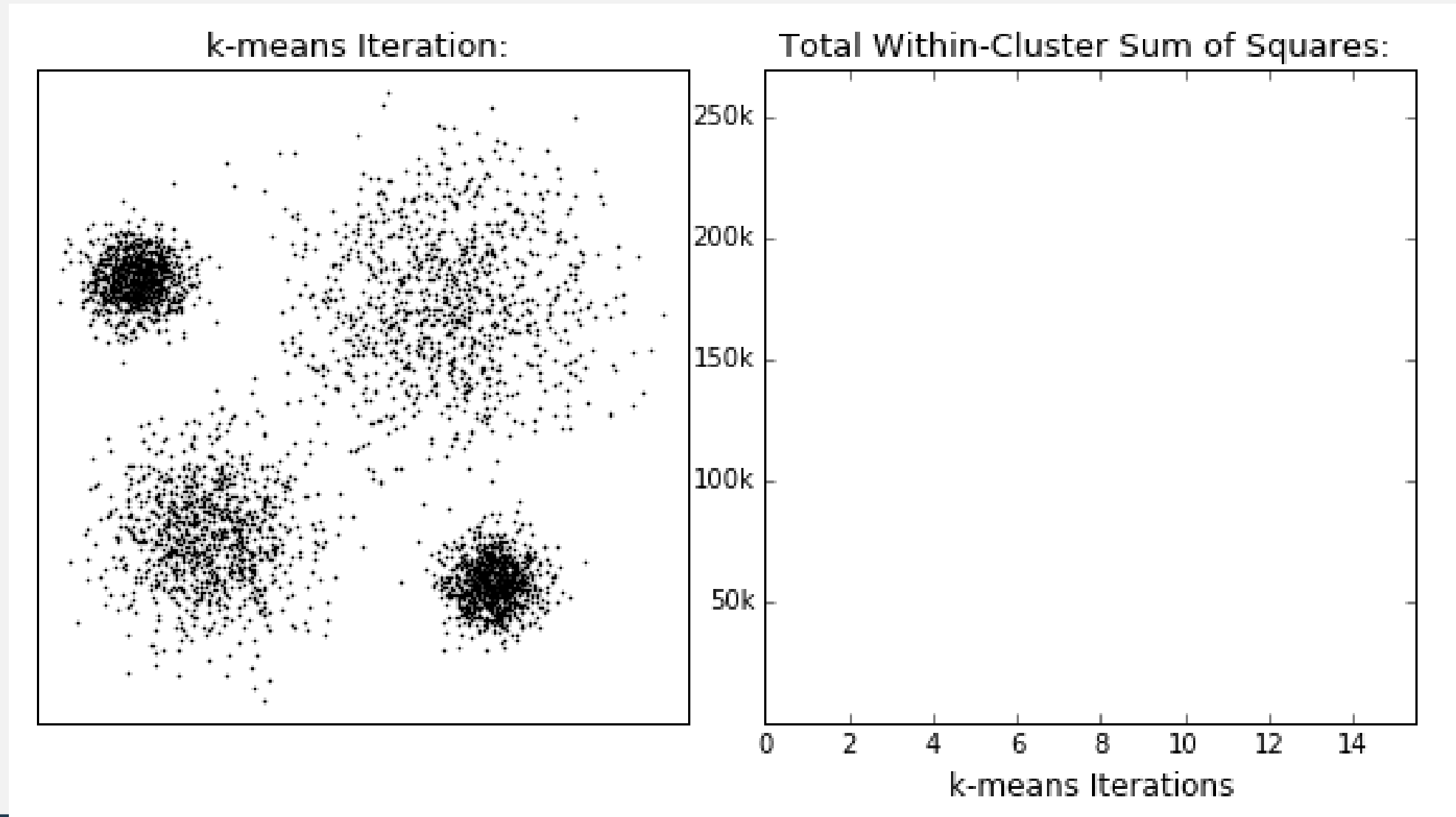


Image
from: [link](#)

K-means Formal [1]

- K-means also known as Lloyd's algorithm is a centroid-based clustering algorithm.
- It has one hyperparameter that is set by the user: the number of clusters K .
- The goal is to divide the instances in S into K disjoint cluster sets C_j . Each cluster C_j has a centroid μ_j which is the mean of the instances in C_j .
- In particular, the K-means seeks to find clusters with centroids that minimize the within-cluster sum-of-squares:

$$\arg \min_{\mu_1, \mu_2, \dots, \mu_K} \sum_{j=1}^K \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$

K-means Formal [2]

$$\arg \min_{\mu_1, \mu_2, \dots, \mu_K} \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

- Solving this exactly is computationally difficult (NP-Hard). So, we opt for a heuristic solution: K-means clustering. It is not guaranteed to find the optimal cluster assignments that minimizes this quantity.
- K-means is an iterative solution that goes as follows:
 1. Randomly initialize the centroids $\mu_1, \mu_2, \dots, \mu_K$
 2. For N iterations or until no assignment change occurs do:
 - **Assignment:** assign each instance $x_i \in S$ to its nearest centroid μ_j . This corresponds to assigning instance x_i to cluster C_j .
 - **Update:** after assigning all instances to clusters, update the centroid vectors:
$$\mu_j := \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

K-means Formal [3]

- K-means is not guaranteed to find the minimum within-cluster sum-of-squares.
- This means that different random initializations can lead to different final clusters.
- How to implement this algorithm?
- We need to maintain the cluster assignments for each instance. We can do this with a vector, and refer to each cluster as an integer. For example, assign x_1 to cluster 4.
- Assignment step computes the Euclidean distance between each cluster centroid and instance. Assign the instance to its nearest centroid (min Euclidean distance).
- Update step uses the cluster-assignment-vector and the instances to update the centroid vectors. A centroid vector becomes the mean of its instances.

K-means Implementation [1]

Randomly initialize K cluster centroids (np.random)

$$\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^d$$

Repeat until no assignment change {

- **Assignment**

for $i = 1$ to m

- $c^{(i)} := \text{index}(\text{from } 1 \text{ to } K) \text{ of cluster centroid closest to } x^{(i)} \text{ (np.argmin)}$

- **Update**

for $k = 1$ to K

- $\mu_k := \text{average (mean) of points assigned to cluster } k \text{ (np.mean and np.where)}$

}

- Runtime complexity is $O(KmdN)$, where K is number of clusters, m number of instances, d number of features, N number of iterations until convergence.

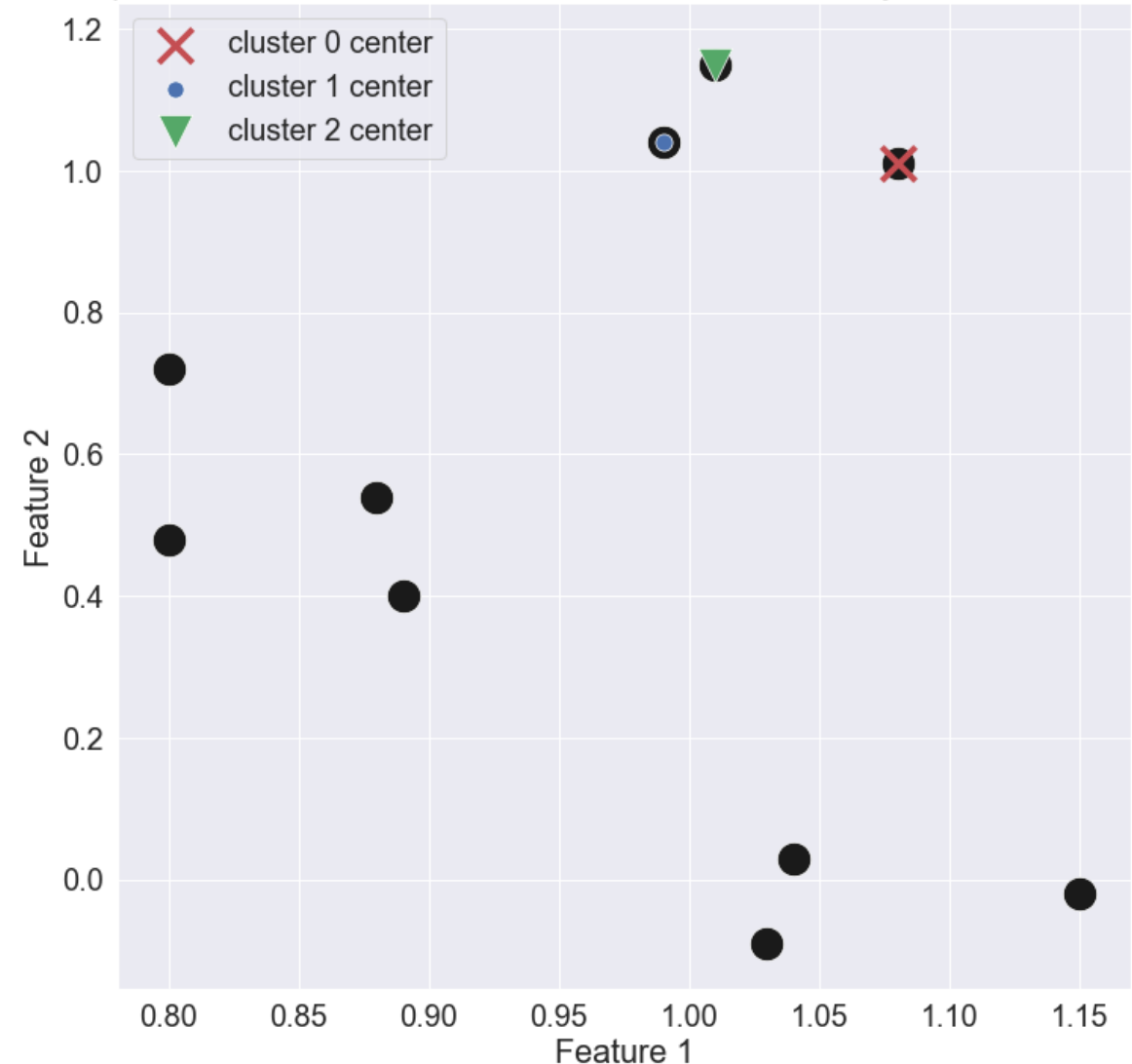
K-means example [1]

Consider the following dataset. We want to apply K-means with $K = 3$. The centroids have been randomly set to one of the datapoints. Apply the iterations of K-means.

feature 1	feature 2
1.15	-0.02
0.80	0.72
1.01	1.15
1.08	1.01
0.80	0.48
0.99	1.04
0.89	0.40
1.04	0.03
0.88	0.54
1.03	-0.09

	feature 1	feature 2
mu_0	1.08	1.01
mu_1	0.99	1.04
mu_2	1.01	1.15

Initial setup of K-means with $K = 3$. Centers have been randomly set as random datapoints.

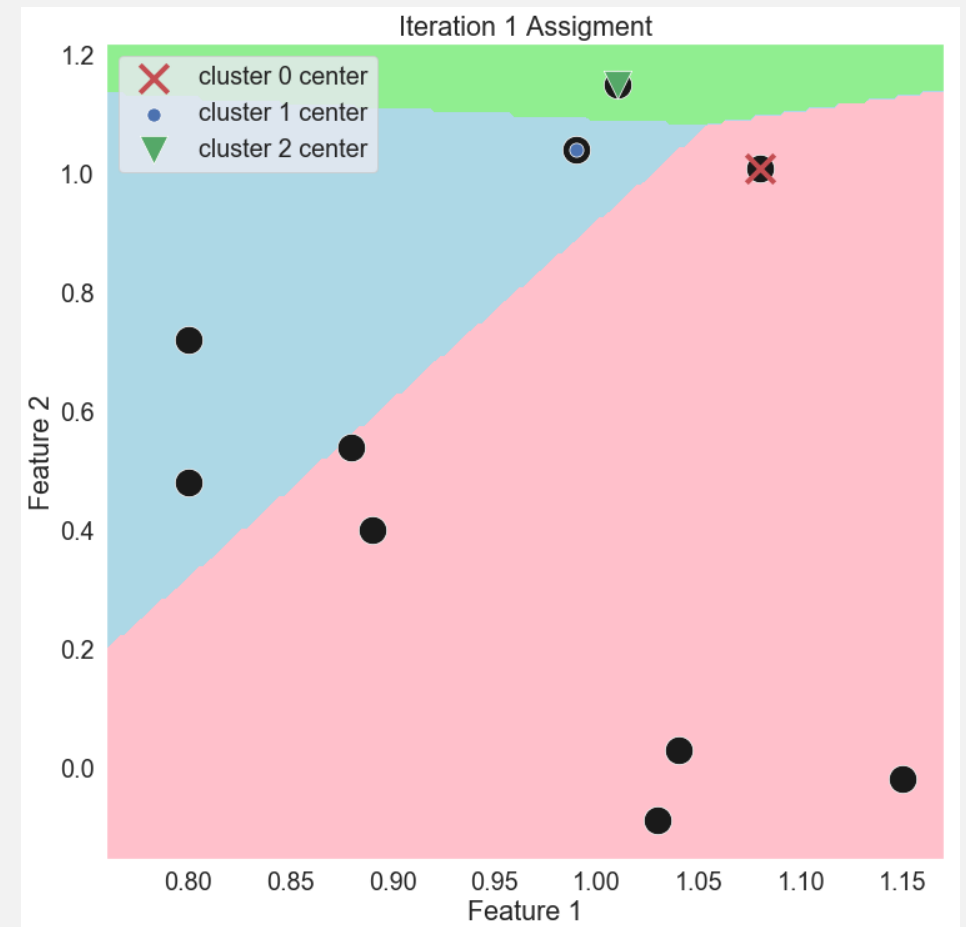


K-means example [1]

Perform iteration 1: Assignment step, then Update centers step.

	feature 1	feature 2
mu_0	1.08	1.01
mu_1	0.99	1.04
mu_2	1.01	1.15

feature 1	feature 2	dist(x,mu_0)	dist(x,mu_1)	dist(x,mu_2)	Cluster Assignment
1.15	-0.02	1.03	1.07	1.18	0
0.80	0.72	0.40	0.37	0.48	1
1.01	1.15	0.16	0.11	0.00	2
1.08	1.01	0.00	0.09	0.16	0
0.80	0.48	0.60	0.59	0.70	1
0.99	1.04	0.09	0.00	0.11	1
0.89	0.40	0.64	0.65	0.76	0
1.04	0.03	0.98	1.01	1.12	0
0.88	0.54	0.51	0.51	0.62	0
1.03	-0.09	1.10	1.13	1.24	0



Using cluster assignments, we can now update centroids to be mean of cluster members

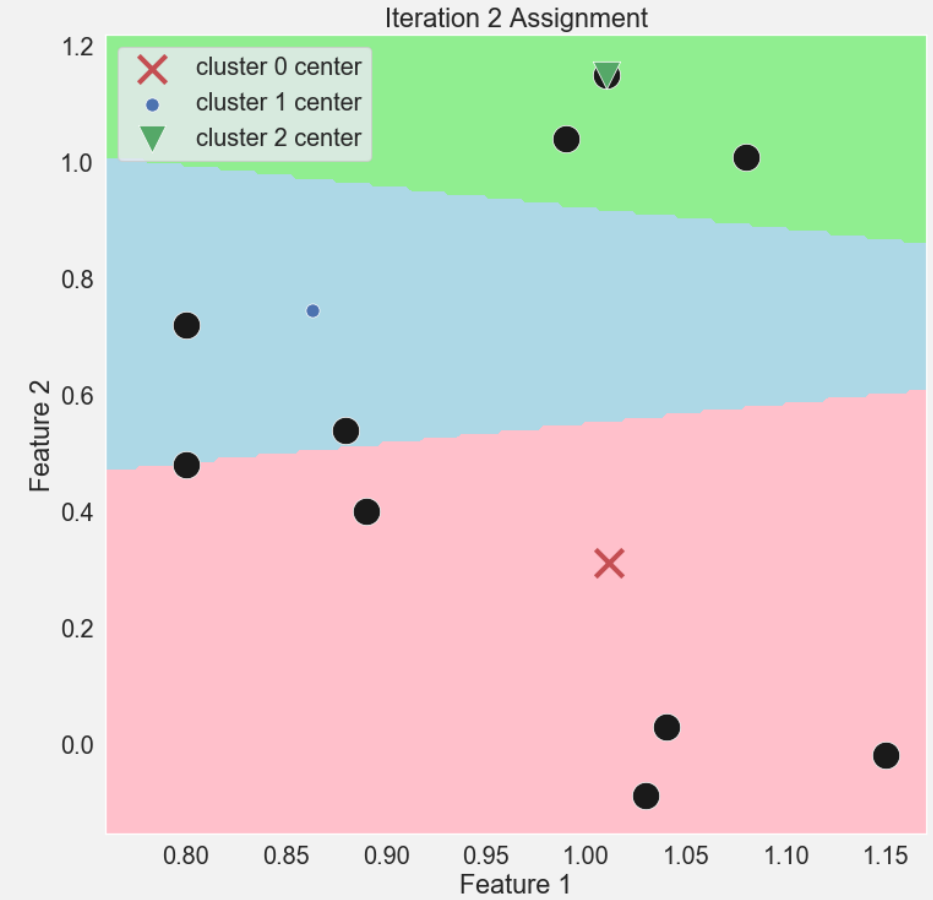


	feature 1	feature 2
mu_0	1.01	0.31
mu_1	0.86	0.75
mu_2	1.01	1.15

K-means example [2]

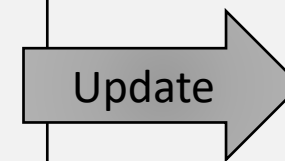
Perform iteration 2: Assignment step, then Update centers step.

	feature 1	feature 2
mu_0	1.01	0.31
mu_1	0.86	0.75
mu_2	1.01	1.15



feature 1	feature 2	dist(x,mu_0)	dist(x,mu_1)	dist(x,mu_2)	Cluster Assignment
1.15	-0.02	0.36	0.82	1.18	0
0.80	0.72	0.46	0.07	0.48	1
1.01	1.15	0.84	0.43	0.00	2
1.08	1.01	0.70	0.34	0.16	2
0.80	0.48	0.27	0.28	0.70	0
0.99	1.04	0.73	0.32	0.11	2
0.89	0.40	0.15	0.35	0.76	0
1.04	0.03	0.28	0.74	1.12	0
0.88	0.54	0.26	0.21	0.62	1
1.03	-0.09	0.40	0.86	1.24	0

Using cluster assignments, we can now update centroids to be mean of cluster members



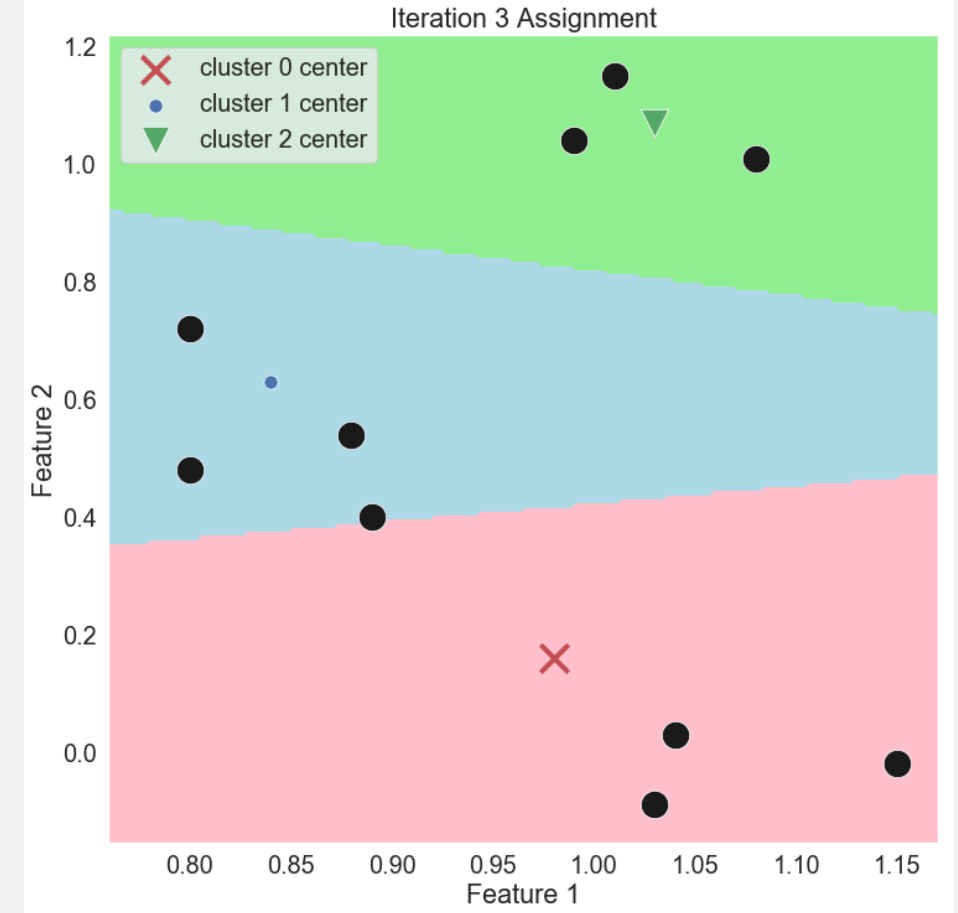
	feature 1	feature 2
mu_0	0.98	0.16
mu_1	0.84	0.63
mu_2	1.03	1.07

K-means example [3]

Perform iteration 3: Assignment step, then Update centers step.

	feature 1	feature 2
mu_0	0.98	0.16
mu_1	0.84	0.63
mu_2	1.03	1.07

feature 1	feature 2	dist(x,mu_0)	dist(x,mu_1)	dist(x,mu_2)	Cluster Assignment
1.15	-0.02	0.25	0.72	1.10	0
0.80	0.72	0.59	0.10	0.42	1
1.01	1.15	0.99	0.55	0.08	2
1.08	1.01	0.86	0.45	0.08	2
0.80	0.48	0.37	0.16	0.63	1
0.99	1.04	0.88	0.44	0.05	2
0.89	0.40	0.26	0.24	0.68	1
1.04	0.03	0.14	0.63	1.04	0
0.88	0.54	0.39	0.10	0.55	1
1.03	-0.09	0.25	0.74	1.16	0



Using cluster assignments, we can now update centroids to be mean of cluster members



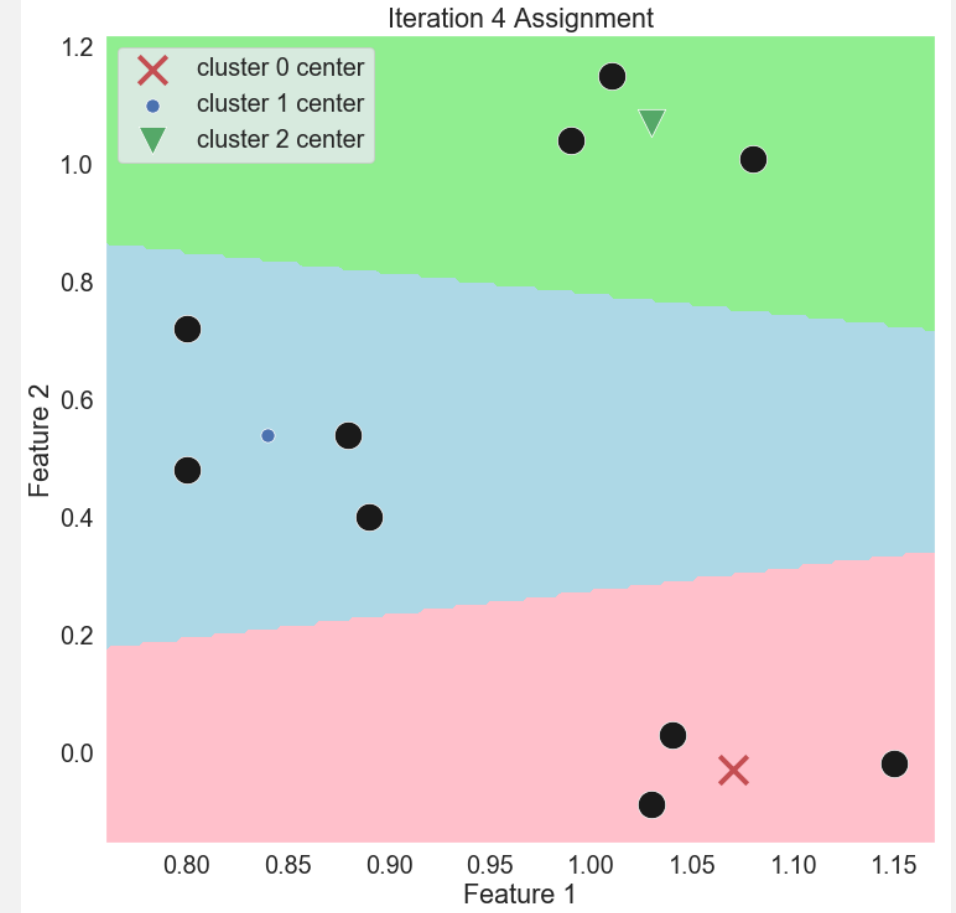
	feature 1	feature 2
mu_0	1.07	-0.03
mu_1	0.84	0.54
mu_2	1.03	1.07

K-means example [4]

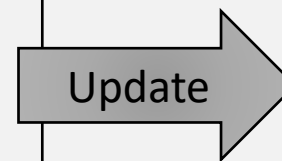
Perform iteration 4: Assignment step, then Update centers step.

	feature 1	feature 2
mu_0	1.07	-0.03
mu_1	0.84	0.54
mu_2	1.03	1.07

feature 1	feature 2	dist(x,mu_0)	dist(x,mu_1)	dist(x,mu_2)	Cluster Assignment
1.15	-0.02	0.08	0.64	1.10	0
0.80	0.72	0.80	0.18	0.42	1
1.01	1.15	1.18	0.63	0.08	2
1.08	1.01	1.04	0.53	0.08	2
0.80	0.48	0.58	0.07	0.63	1
0.99	1.04	1.07	0.52	0.05	2
0.89	0.40	0.47	0.15	0.68	1
1.04	0.03	0.07	0.55	1.04	0
0.88	0.54	0.60	0.04	0.55	1
1.03	-0.09	0.07	0.66	1.16	0



Assignment and updates have converged (stopped updating), so we can stop here.



	feature 1	feature 2
mu_0	1.07	-0.03
mu_1	0.84	0.54
mu_2	1.03	1.07

K-means Questions [1]

$$\arg \min_{\mu_1, \mu_2, \dots, \mu_K} \sum_{j=1}^K \sum_{x_i \in C_j} ||x_i - \mu_j||^2$$

- K-means is not guaranteed to find the minimum within-cluster sum-of-squares (i.e. optimal cluster assignment).
- Different random initializations can lead to different final cluster assignments.
- However, given a random initialization, the K-means algorithm converges to a *local* optimal solution. *Local* here means for the given random initialization. This local may or may not be the global optimal.
- For a given K , how to increase chance of finding global optimal minima?
 - Can re-run many times with different initializations and select the clustering which gives minimum within-cluster sum-of-squares.
- How do we choose K (the number of clusters)?
 - K could be determined by the specific task. For example, grouping documents into 5 science fields.
 - Elbow method: step-wise increase K until within-cluster sum-of-squares does not decrease drastically.

K-means in scikit-learn [1]

```
class sklearn.cluster.KMeans(n_clusters=8, *, init='k-means++',  
n_init='auto', max_iter=300, tol=0.0001, verbose=0, random_state=None,  
copy_x=True, algorithm='Lloyd') \[source\]
```

```
1  from sklearn.cluster import KMeans  
2  
3  kmeans = KMeans(n_clusters=5, random_state=0)  
4  kmeans.fit(X)  
5  
6  kmeans.labels_ | # returns cluster assignment for X  
7  
8  kmeans.cluster_centers_ # returns cluster centroids  
9  
10 kmeans.predict(x_new) # assigns x_new to nearest centroid cluster
```

K-means random initializations

- As mentioned, initialization of centroids matters and will affect the resulting clustering.
- In scikit-learn there is a parameter for this:

`init : {'k-means++', 'random'}, callable or array-like of shape (n_clusters, n_features), default='k-means++'`

Method for initialization:

'k-means++': selects initial cluster centers for k-mean clustering in a smart way to speed up convergence. See section Notes in `k_init` for more details.

'random': choose `n_clusters` observations (rows) at random from data for the initial centroids.

- There is also a parameter for the number of re-runs:

`n_init : int, default=10`

Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of `n_init` consecutive runs in terms of inertia.

- Inertia here is another word for within-cluster sum-of-squares.

Elbow method to select K in K-means

- To select K, you can run K-means as you increase K.
- Plot the within-cluster sum-of-squares vs K. The point at which an elbow forms is where you can set your K.
- The elbow is the point at which increasing K doesn't get you a significant decrease in the squared error. In the image to the right, setting K to 3, 4 or 5 is reasonable.

```
1 for k in range(1, 10):  
2     kmeans = KMeans(n_clusters=k)  
3     kmeans.fit(X)  
4     sse[k] = kmeans.inertia_  
5
```

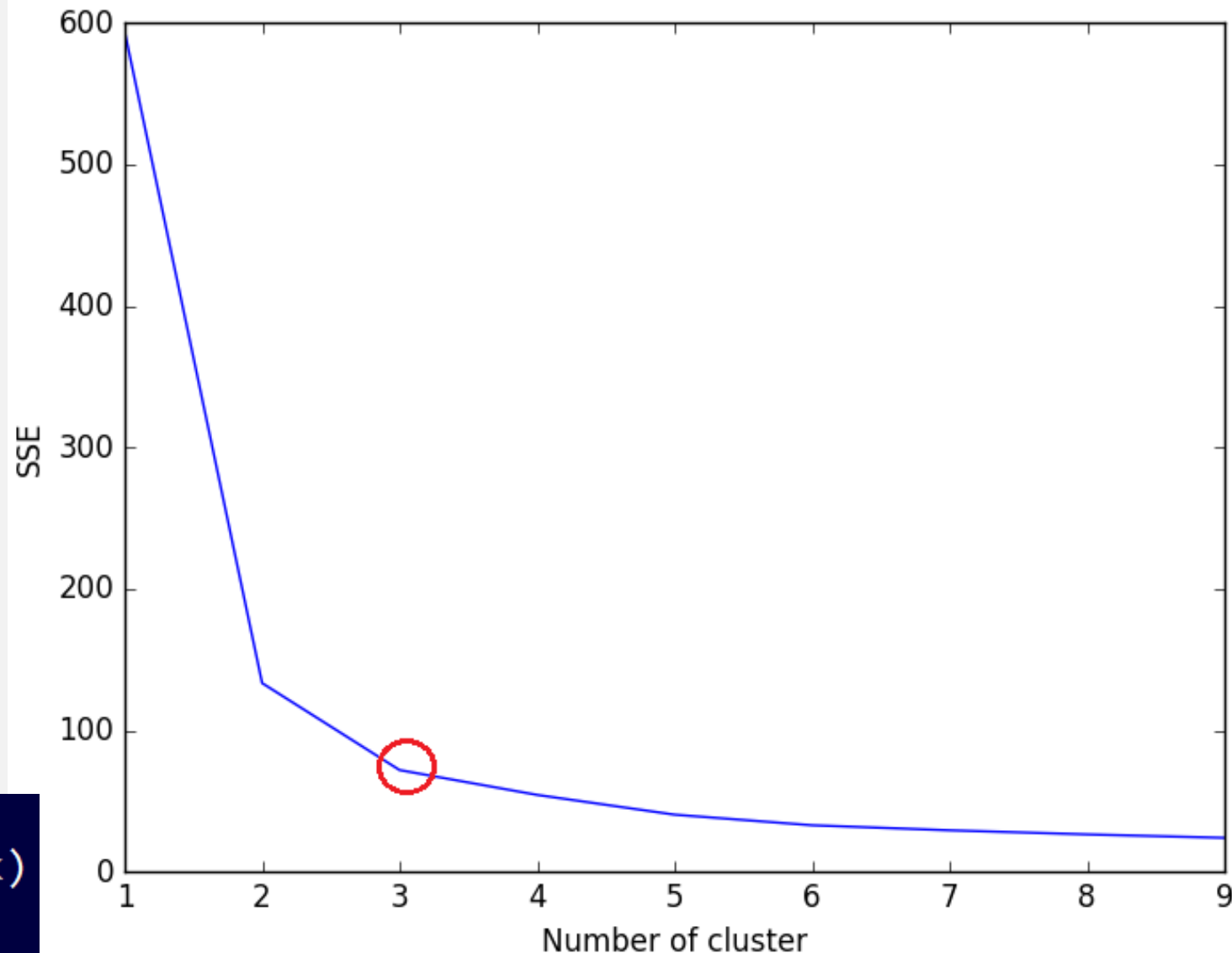


Image from: [link](#)

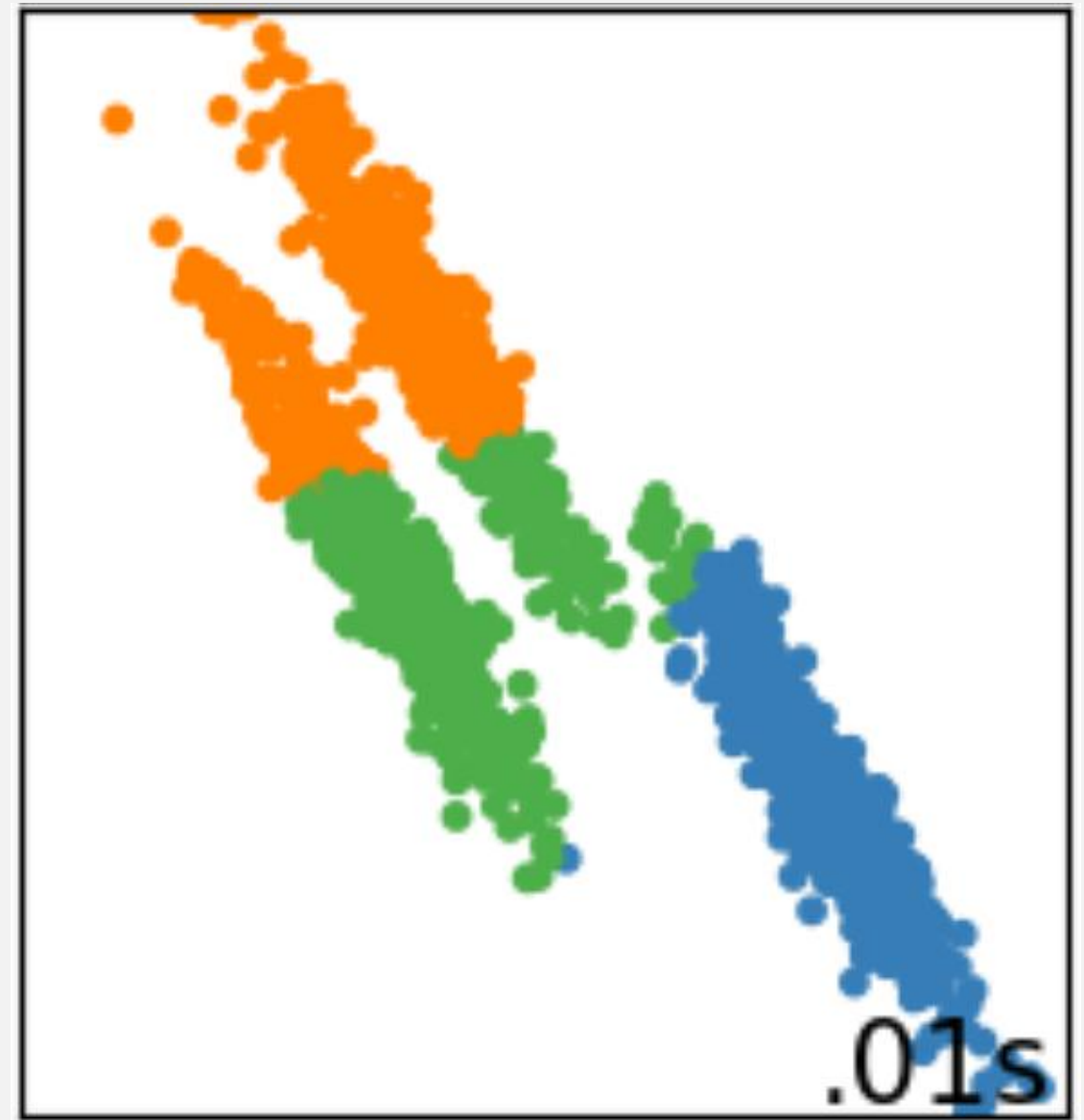
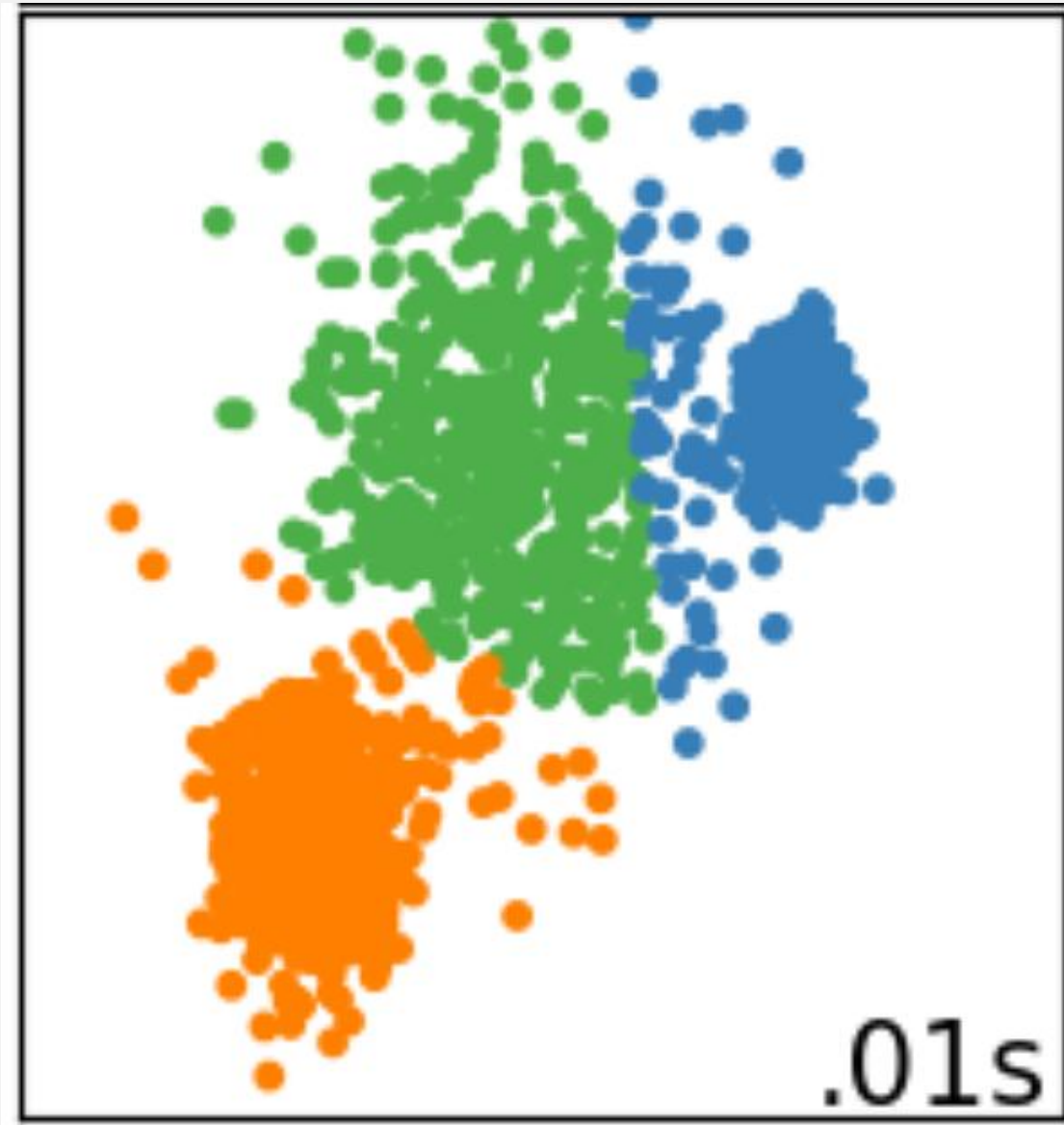
K-means Strengths and Weaknesses

Strengths:

- Easy to implement.
- Scales well to large datasets. Runtime complexity is linear in problem parameters. $O(KNmd)$, where K is number of clusters, m number of instances, d number of features, N number of iterations until convergence.
- Produces tight spherical clusters for the data.

Weaknesses:

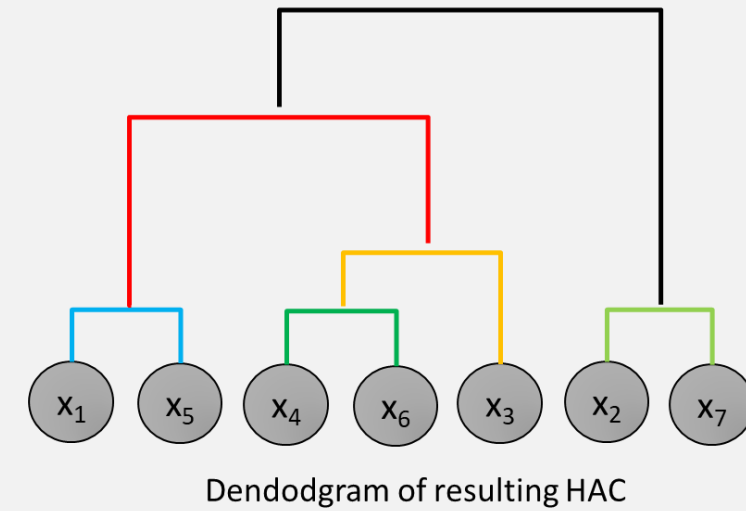
- Need to define number of clusters.
- K-means uses Euclidean distance (due to it using centroids). Other variants exist but are not called k-means ([K-medoids](#) that can be used for general distance metric).
- Random initialization of centroids affects the final clustering. May require multiple re-runs to find a better (lower) within-cluster sum-of-squares.
- Has trouble clustering data of varying blob sizes and density. (see examples next slide and also at [link](#))
- Outliers can drag the centroids.



Hierarchical Clustering

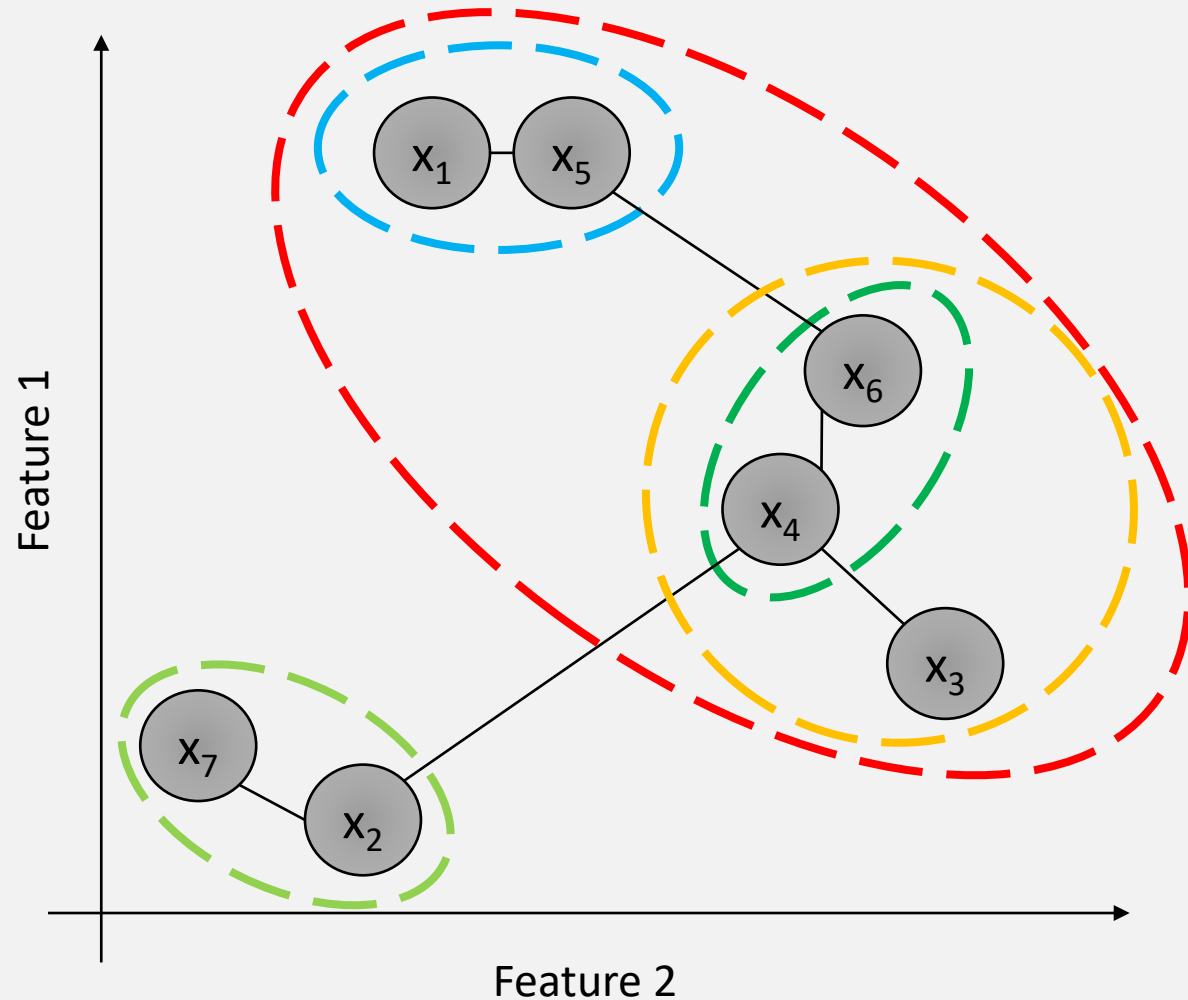
Hierarchical Clustering Overview [1]

- Hierarchical clustering clusters instances in a hierarchy of clusters. This is represented as a tree-based structure called a dendrogram.

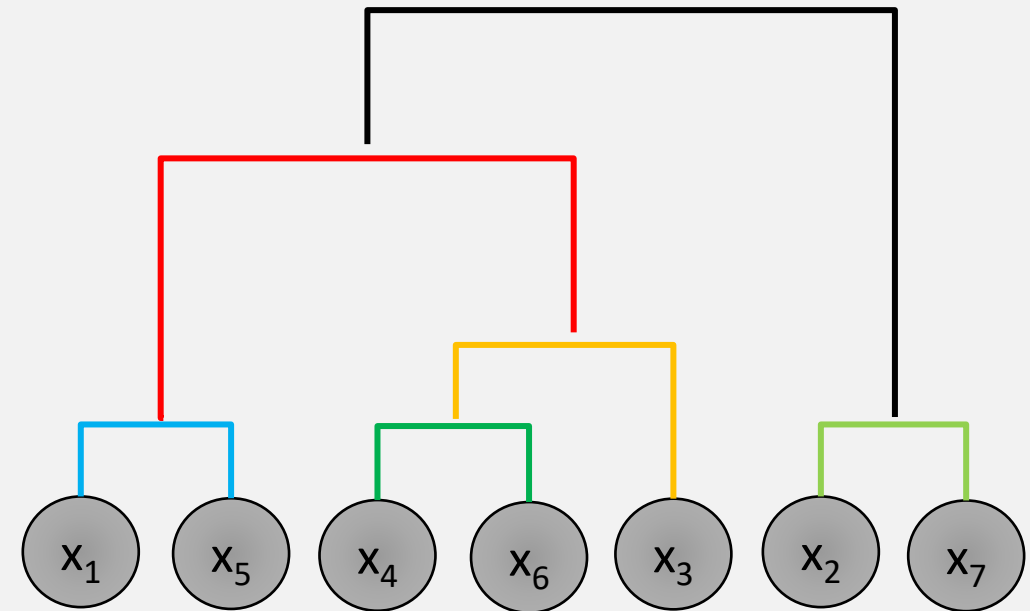


- Two approaches:
 - Agglomerative: bottom-up approach where each instance starts in its own cluster, then we merge clusters that are near each other in distance. So, we go up the hierarchy as we merge.
 - Divisive: top-down approach where all instances are in one cluster, then we split clusters based on distances until each instance is in its own cluster. So, we go down the hierarchy as we split.
- We will discuss Hierarchical Agglomerative Clustering (HAC) in this class.
- Advantages:
 - can use any distance measure.
 - After building dendrogram, can easily define various cluster assignment using distance threshold

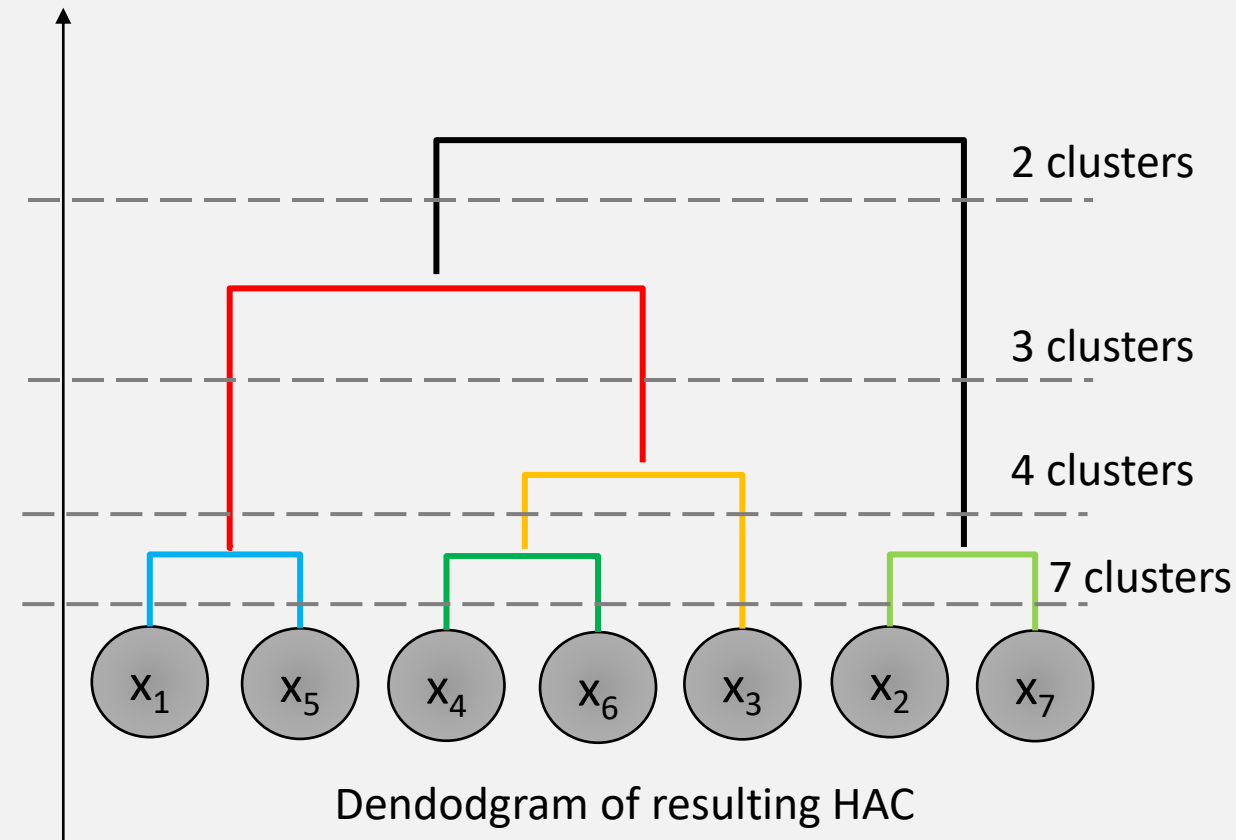
- Consider the two-feature dataset. HAC builds clusters bottom-up. The algorithm proceeds by **merging** clusters that are nearest each other. At the beginning, instances start in their own clusters.
- The resulting clustering can be visualized as a dendrogram. Links represents clusters, and height represents distance.



Height represents distance between links. So as you go up the dendrogram, distance between link increases.



- Distance thresholding the dendrogram gives final cluster assignments.
- Can visually inspect the dendrogram and define suitable threshold.
- Can try possible thresholds and compute performance measure on resulting clusters. Pick threshold with best performance.

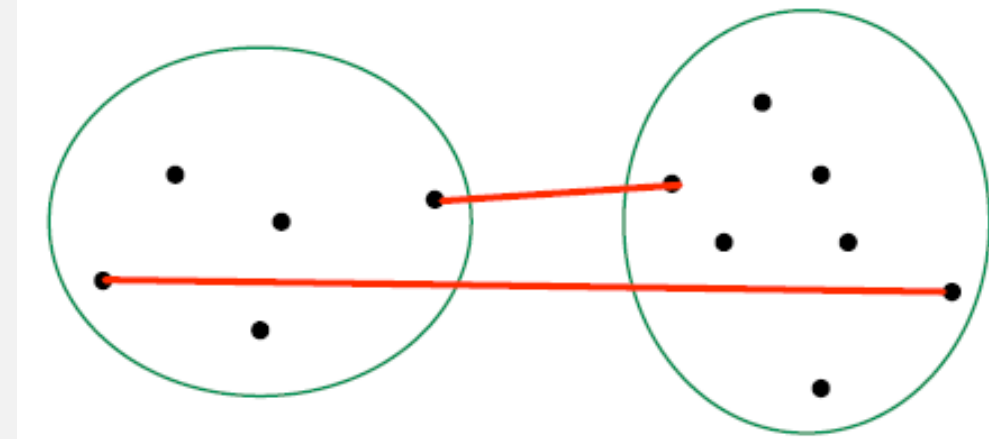


HAC Algorithm

- In HAC, a general algorithm can be written as follows:
 1. Given a dataset $\{x_1, x_2, \dots, x_m\}$, put each instance in its own cluster. So, at the start we have m clusters $C_i = \{x_i\}$. Create a tree TR with the clusters as leaves. Create a list of the current clusters $l = \{C_1, C_2, \dots, C_m\}$.
 2. Until there is a single cluster in l containing all instances, do:
 3. Among the current cluster in l , find two clusters $C_i \in l$ and $C_j \in l$, that have minimum **cluster distance** to each other.
 4. Replace C_i and C_j with $C_i \cup C_j$ in l . Also, add the parent node $C_i \cup C_j$ to the tree TR .
 5. Return the tree TR which represents the dendrogram of the clustering.
- Runtime is $O(m^3)$. There are optimized implementations that are $O(m^2)$. Here m is the number of instances.

HAC Linkage [1]

- We saw in previous example, HAC as it proceeds will merge clusters. So, a cluster distance method needs to be defined to measure distance between two clusters.



- Three common methods:
 - **Single-Linkage:** uses **min** distance between instances in the two clusters.

$$dist(C_i, C_j) = \min_{x_a \in C_i, x_b \in C_j} dist(x_a, x_b)$$

- **Complete-Linkage:** uses **max** distance between instances in the two clusters.

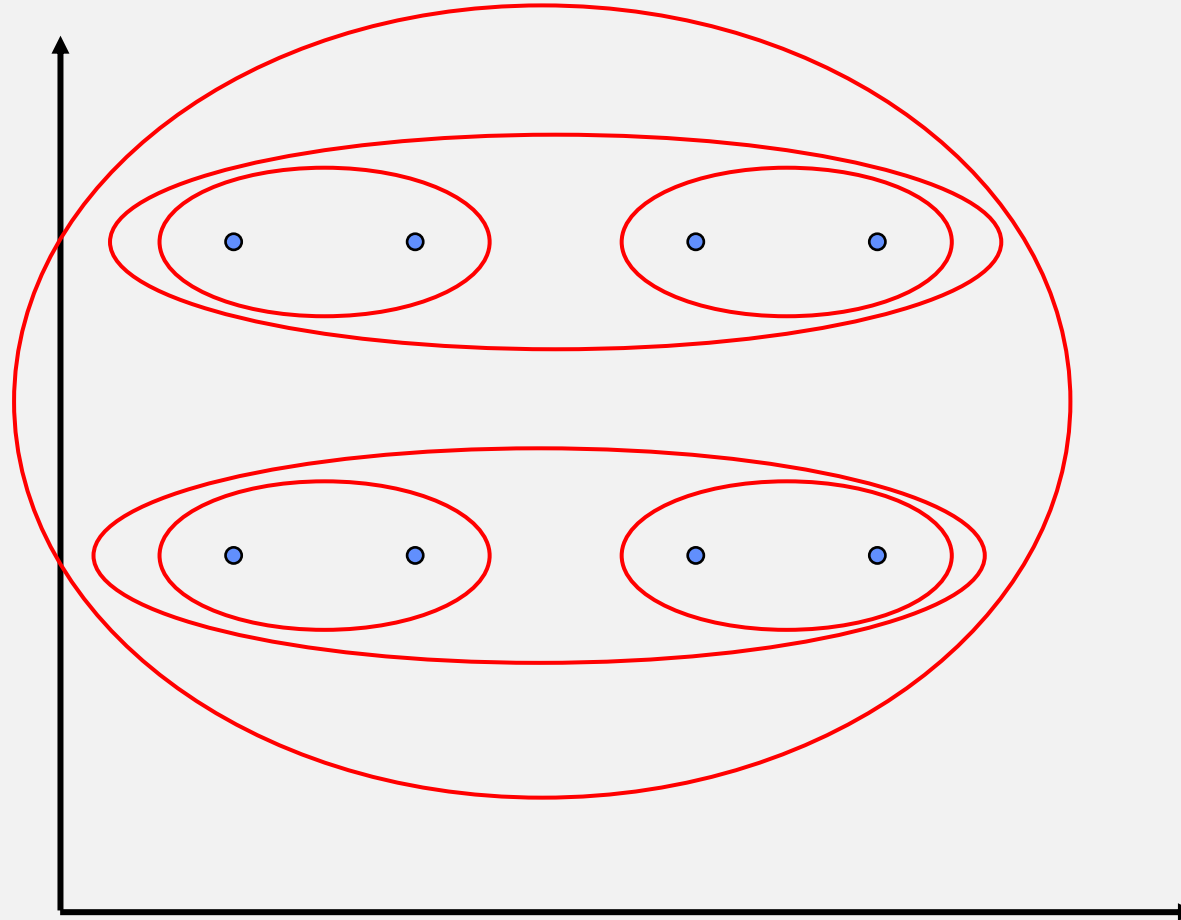
$$dist(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} dist(x_a, x_b)$$

- **Average-Linkage:** uses **avg** distance between instances in the two clusters.

$$dist(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{x_a \in C_i} \sum_{x_b \in C_j} dist(x_a, x_b)$$

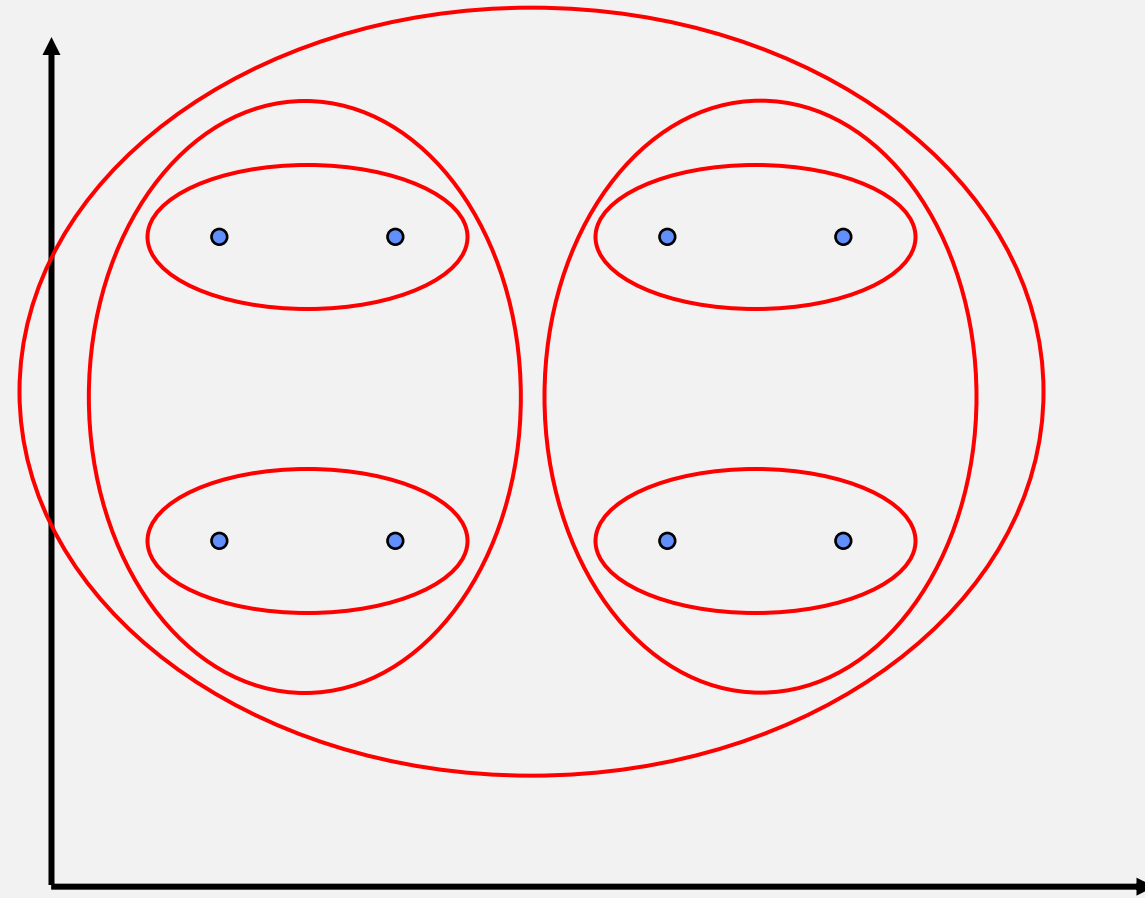
Single-linkage vs complete-linkage [1]

- Single-linkage can result in long and thin clusters.



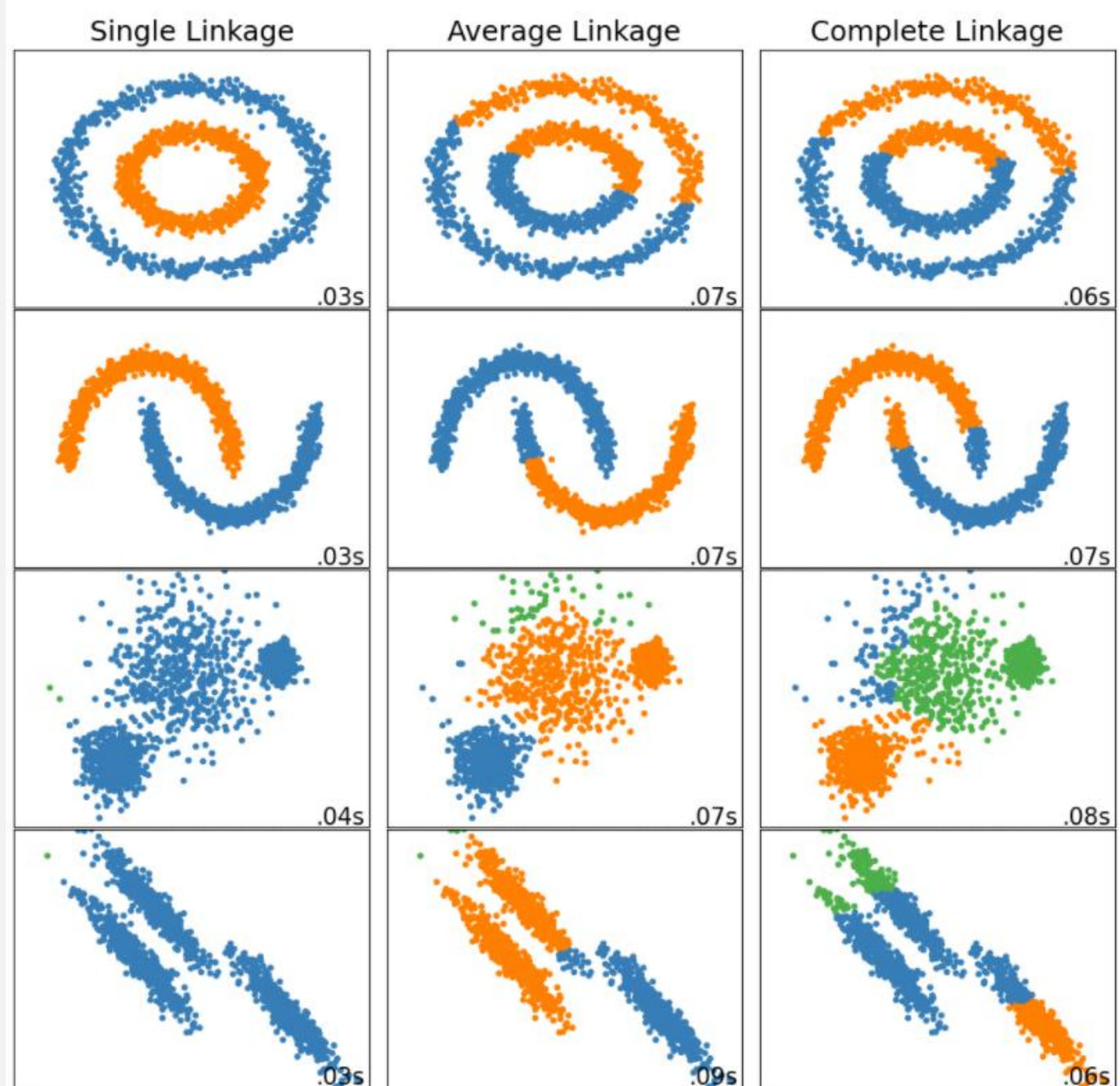
Single-linkage vs complete-linkage [2]

- Complete-linkage results in tight spherical clusters.



HAC Linkage [2]

- Single-linkage can result in long and thin clusters.
- Complete-linkage results in tight spherical clusters.
- Average-linkage is the middle ground between single and complete linkage.



HAC example [1]

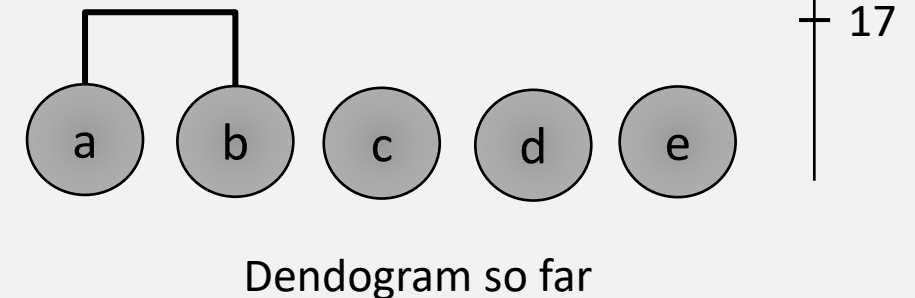
Consider if we have 5 data points (a, b, c, d, e). We want to apply HAC with **complete-linkage** and build the dendrogram. The following is the dataset distance matrix.

$$dist(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} dist(x_a, x_b)$$

	a	b	c	d	e
a	0	17	21	31	23
b	17	0	30	34	21
c	21	30	0	28	39
d	31	34	28	0	43
e	23	21	39	43	0

This is the current distance matrix, each point belongs in its own cluster. We find the two clusters with minimal distance. In this case, it is a and b.

So, we merge a and b into a single cluster.



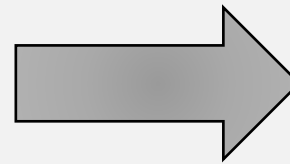
Example from: [link](#)

HAC example [2]

$$\text{dist}(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} \text{dist}(x_a, x_b)$$

Now replace the rows for a and b with a single row (a,b).

	a	b	c	d	e
a	0	17	21	31	23
b	17	0	30	34	21
c	21	30	0	28	39
d	31	34	28	0	43
e	23	21	39	43	0



	(a,b)	c	d	e
(a,b)	0	30	34	23
c	30	0	28	39
d	34	28	0	43
e	23	39	43	0

With complete-linkage, we are looking for the max dist.

$$D_2((a, b), c) = \max(D_1(a, c), D_1(b, c)) = \max(21, 30) = 30$$

$$D_2((a, b), d) = \max(D_1(a, d), D_1(b, d)) = \max(31, 34) = 34$$

$$D_2((a, b), e) = \max(D_1(a, e), D_1(b, e)) = \max(23, 21) = 23$$

Example from: [link](#)

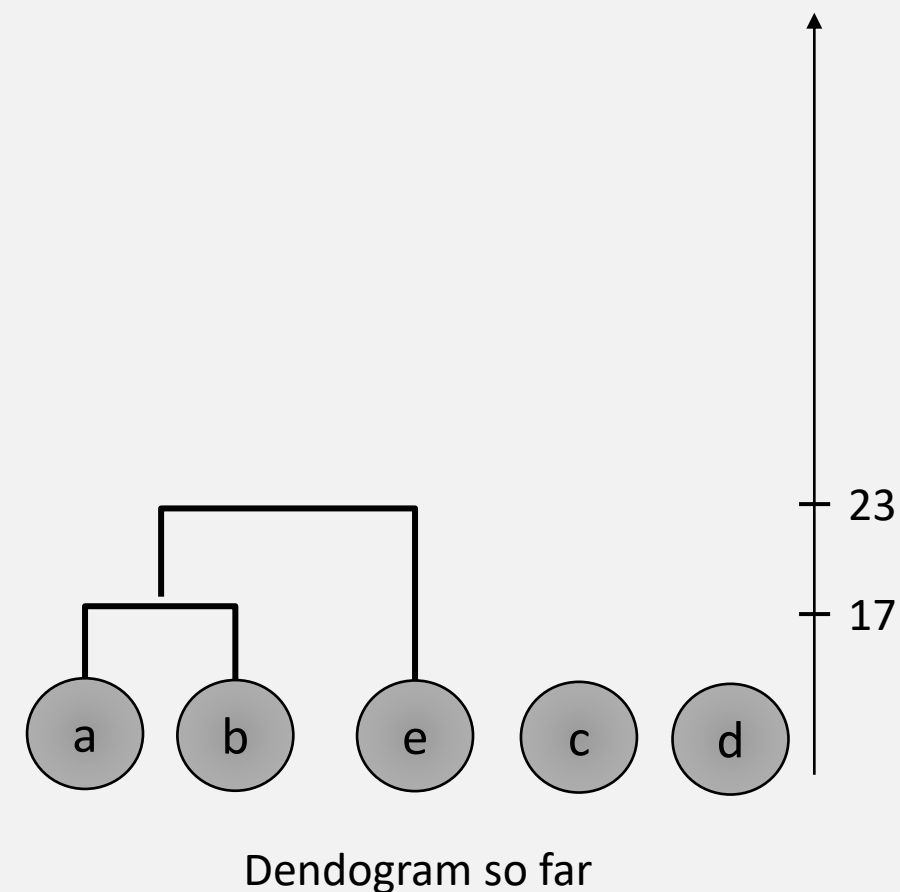
HAC example [3]

$$\text{dist}(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} \text{dist}(x_a, x_b)$$

	(a,b)	c	d	e
(a,b)	0	30	34	23
c	30	0	28	39
d	34	28	0	43
e	23	39	43	0

This is the current distance matrix. We find the two clusters with minimal distance. In this case, it is (a, b) and e.

So, we merge (a, b) and e into a single cluster.



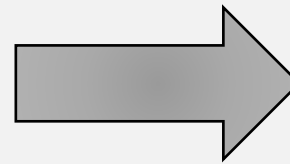
Example from: [link](#)

HAC example [4]

$$dist(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} dist(x_a, x_b)$$

Now replace the rows for (a, b) and e with a single row ((a,b), e).

	(a,b)	c	d	e
(a,b)	0	30	34	23
c	30	0	28	39
d	34	28	0	43
e	23	39	43	0



	((a,b),e)	c	d
((a,b),e)	0	39	43
c	39	0	28
d	43	28	0

With complete-linkage, we are looking for the max dist.

$$D_3(((a, b), e), c) = \max(D_2((a, b), c), D_2(e, c)) = \max(30, 39) = 39$$

$$D_3(((a, b), e), d) = \max(D_2((a, b), d), D_2(e, d)) = \max(34, 43) = 43$$

Example from: [link](#)

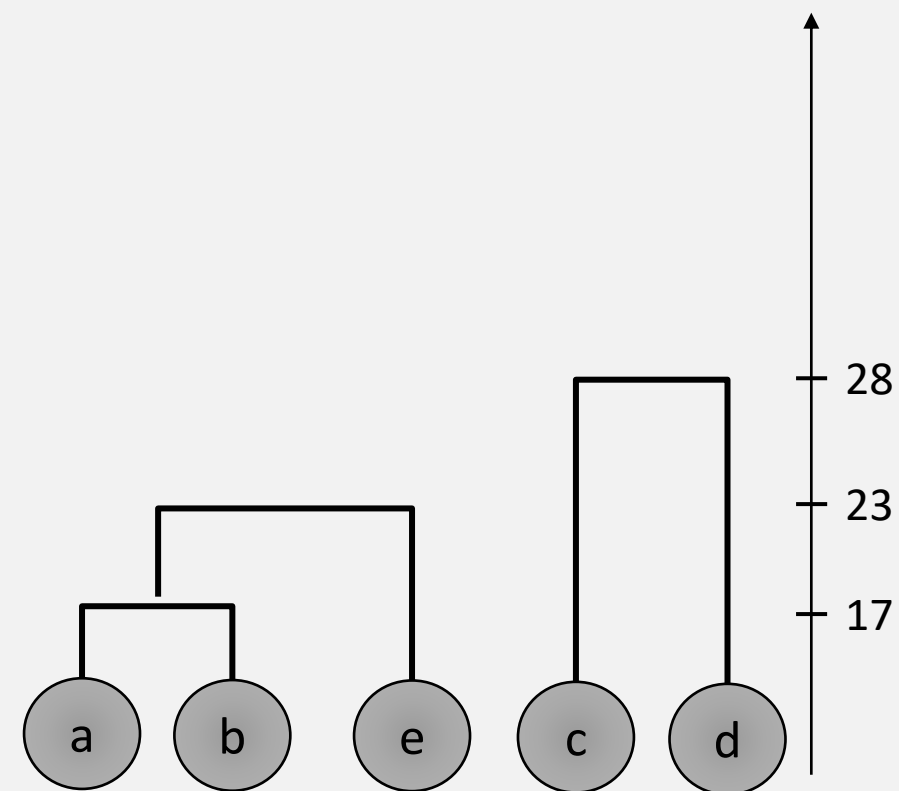
HAC example [5]

$$\text{dist}(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} \text{dist}(x_a, x_b)$$

	((a,b),e)	c	d
((a,b),e)	0	39	43
c	39	0	28
d	43	28	0

This is the current distance matrix. We find the two clusters with minimal distance. In this case, it is c and d.

So, we merge c and d into a single cluster.



Dendrogram so far

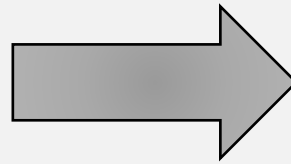
Example from: [link](#)

HAC example [6]

$$\text{dist}(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} \text{dist}(x_a, x_b)$$

Now replace the rows for c and d with a single row (c, d).

	((a,b),e)	c	d
((a,b),e)	0	39	43
c	39	0	28
d	43	28	0



	((a,b),e)	(c,d)
((a,b),e)	0	43
(c,d)	43	0

With complete-linkage, we are looking for the max dist.

$$D_4((c, d), ((a, b), e)) = \max(D_3(c, ((a, b), e)), D_3(d, ((a, b), e))) = \max(39, 43) = 43$$

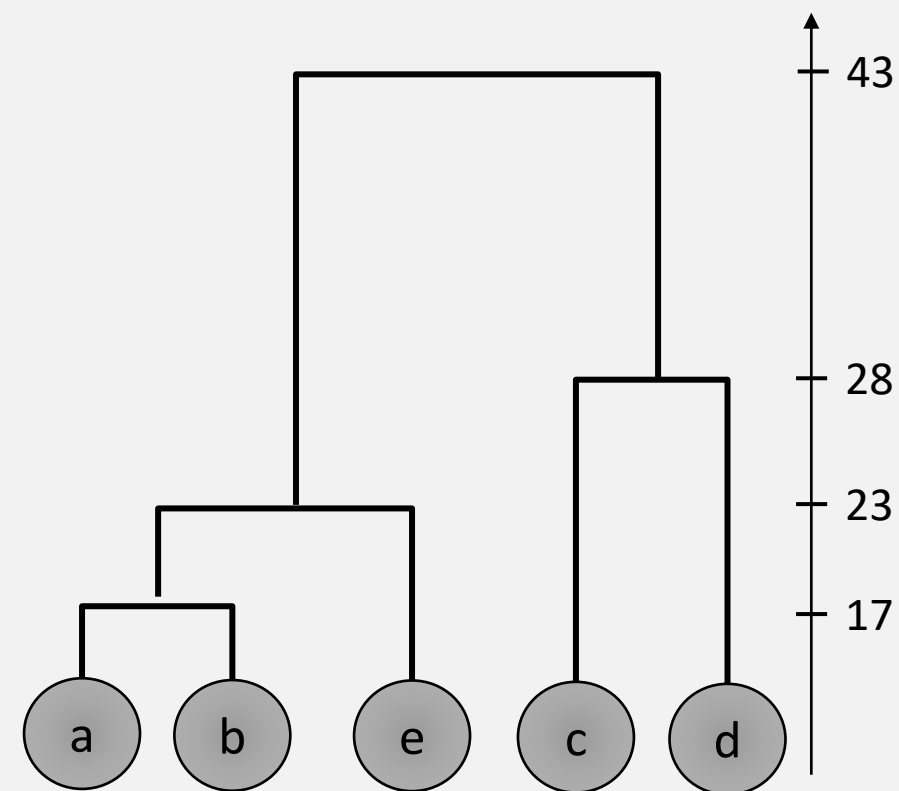
Example from: [link](#)

$$\text{dist}(C_i, C_j) = \max_{x_a \in C_i, x_b \in C_j} \text{dist}(x_a, x_b)$$

	((a,b),e)	(c,d)
((a,b),e)	0	43
(c,d)	43	0

We only have two clusters remaining so we merge them.

Practice: try doing this example with single-linkage.



Dendrogram using HAC with complete-linkage

Example from: [link](#)

HAC in scikit-learn [1]

```
class sklearn.cluster.AgglomerativeClustering(n_clusters=2, *,
metric='euclidean', memory=None, connectivity=None,
compute_full_tree='auto', linkage='ward', distance_threshold=None,
compute_distances=False) # \[source\]
```

linkage : {'ward', 'complete', 'average', 'single'}, default='ward'

Which linkage criterion to use. The linkage criterion determines which distance to use between sets of observation. The algorithm will merge the pairs of cluster that minimize this criterion.

- 'ward' minimizes the variance of the clusters being merged.
- 'average' uses the average of the distances of each observation of the two sets.
- 'complete' or 'maximum' linkage uses the maximum distances between all observations of the two sets.
- 'single' uses the minimum of the distances between all observations of the two sets.

```
1 from sklearn.cluster import AgglomerativeClustering
2
3 clustering = AgglomerativeClustering()
4 clustering.fit(X)
5
6 clustering.labels_ # returns cluster assignment for X
```

HAC in scikit-learn [2]

```
1 # setting distance_threshold=0 ensures we compute the full tree.
2 clustering = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
3
4 fit_clustering = clustering.fit(X)
5 plt.title("Hierarchical Clustering Dendrogram")
6 # plot the top three levels of the dendrogram
7 plot_dendrogram(fit_clustering, truncate_mode="level", p=3)
8 plt.xlabel("Number of points in node (or index of point if no parenthesis).")
9 plt.show()
```

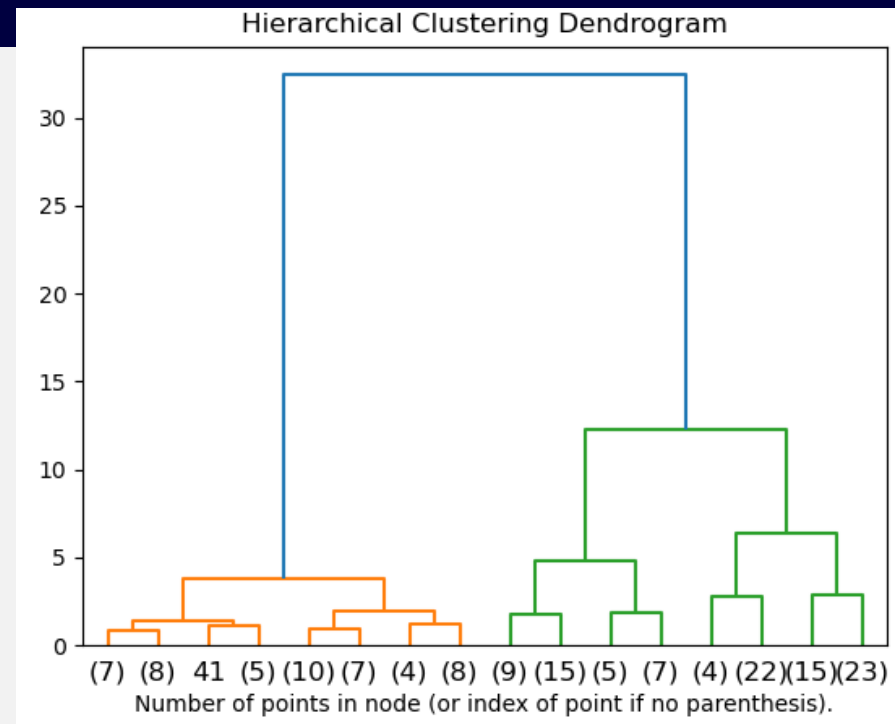


Image from [example](#)

HAC Strengths and Weaknesses

Strengths:

- No need to define number of clusters.
- Can visualize the clustering in a dendrogram. May be able to draw human conclusions based on this.
- Any distance metric can be used.
- Easy to implement.

Weaknesses:

- Runtime is $O(m^3)$ where m is number of instances, optimized implementations are $O(m^2)$. K-means more attractive for massive datasets.
- Need to select appropriate linkage. Each linkage has its drawbacks. Single-linkage has the *chaining* effect. Complete-linkage gives tight spherical clusters.

HAC example to identify molecule clusters

- Example of HAC to cluster molecules.
- A chemist expert can then look at the dendrogram and identify a suitable threshold (the horizontal red dashed line).
- Here they cluster molecules into six classes. Each class can be represented by a reference structure.

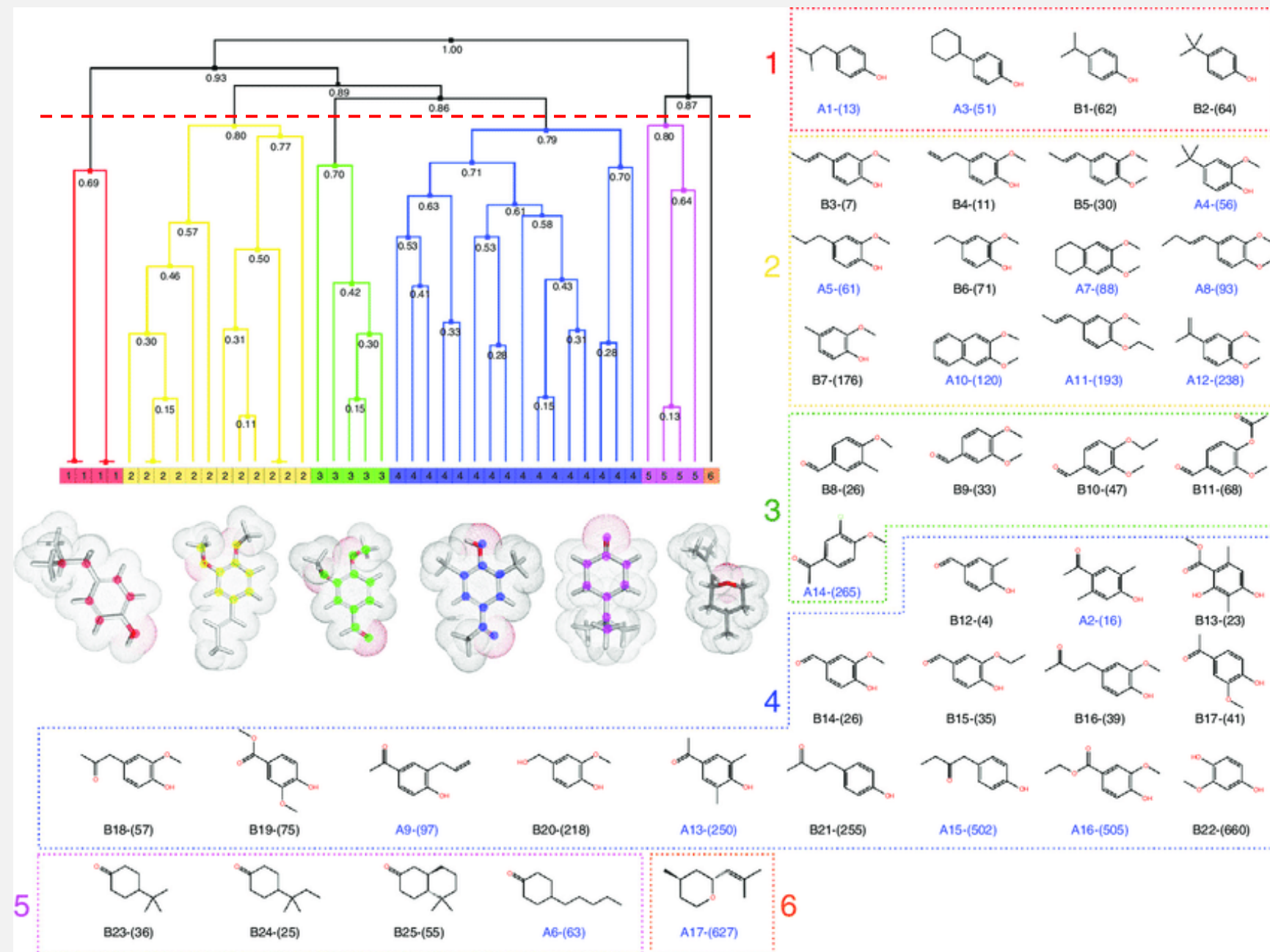
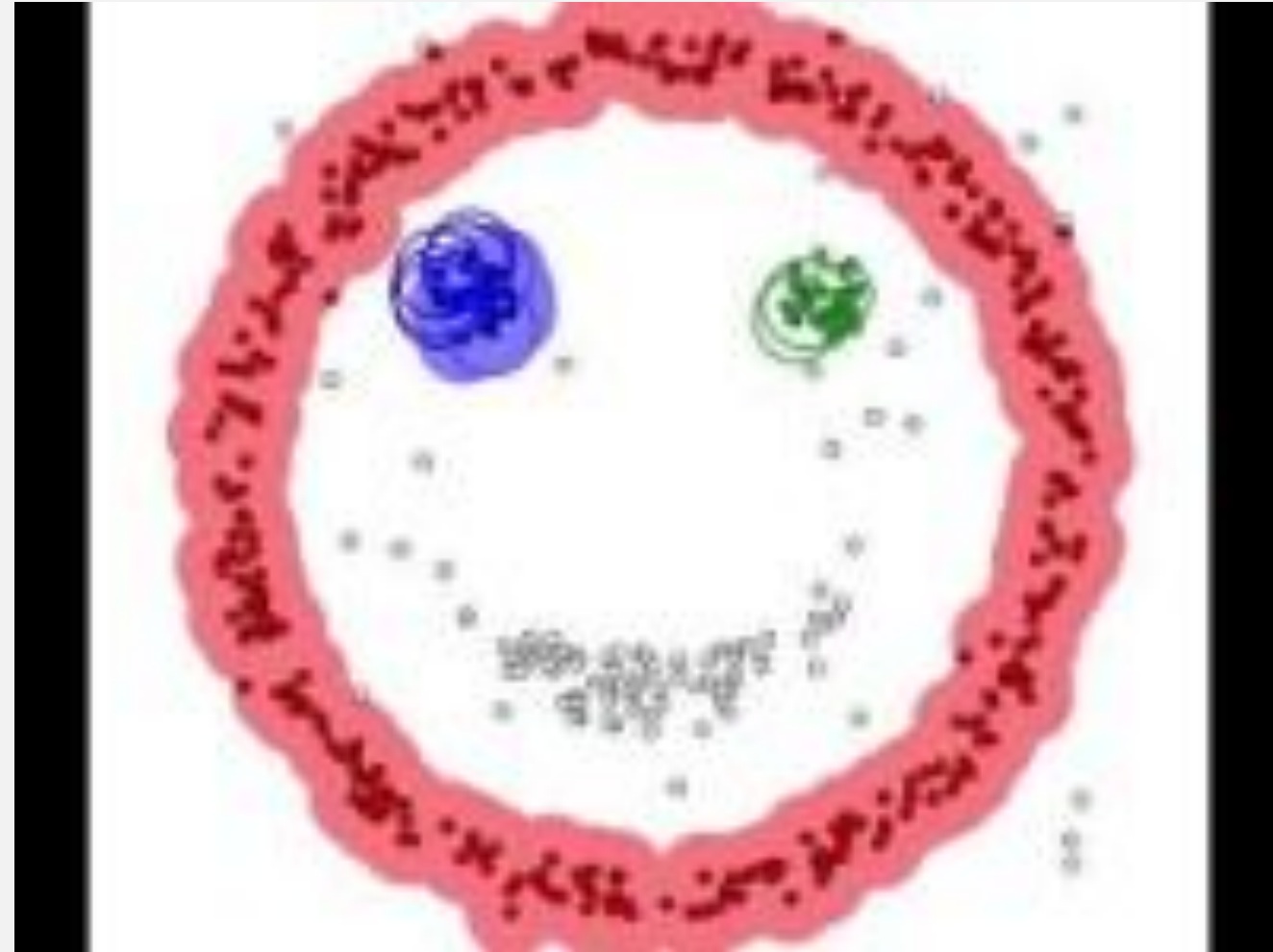


Image from: [link](#)

DBSCAN

DBSCAN Overview [1]

- DBSCAN is a density-based clustering method.
- It can define clusters of arbitrary shape.
- It clusters an instance based on its neighborhood. Instances that are tightly packed together (dense) belong to the same cluster.
- It does this via: **core points** and threshold on neighborhood distance **eps**.
- See the animation of DBSCAN as it clusters the given dataset.
- Full name: **Density-Based Spatial Clustering of Applications with Noise**

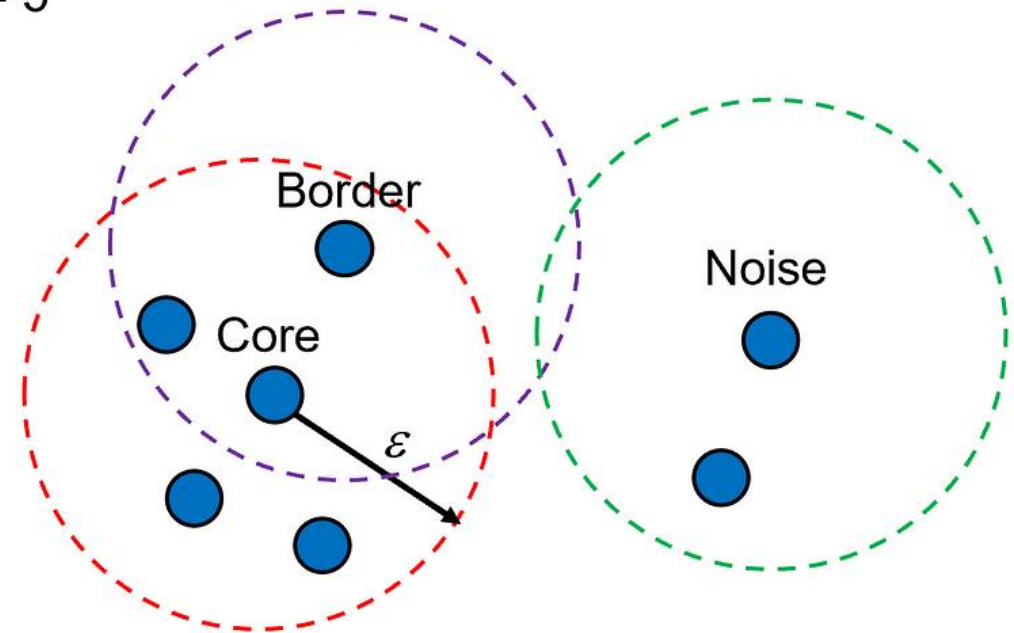


Video from: [link](#)

DBSCAN Concepts – Neighborhood and Core Point

- Two hyperparameters:
 - **eps**: max radius of a neighborhood.
 - **minPts**: min number of neighborhood points required to define a **core point**.
- We say that a point q is in the neighborhood of point p , if the $\text{dist}(p, q) \leq \text{eps}$.
- Thus, the eps-neighborhood of a point p is: $\text{Neighborhood}(p) = \{q : \text{dist}(p, q) \leq \text{eps}\}$
- A point p is a **core point** if:
 $|\text{Neighborhood}(p)| \geq \text{minPts}.$

MinPts = 5



DBSCAN Concepts – Core Point, border point, outlier point

- DBSCAN starts with a random point p , and computes its $Neighborhood(p)$, if p is a **core point**, then it visits its neighbors to see if they are core points.
- The yellow points are **border points**, they are reachable from core points, but are not themselves core points.
- Below we see that the blue point is **noise (outlier points)**. These are points that are not core points and not border points.
- Only core points can be used to *reach* other points. So, a border point cannot be used to reach other points.

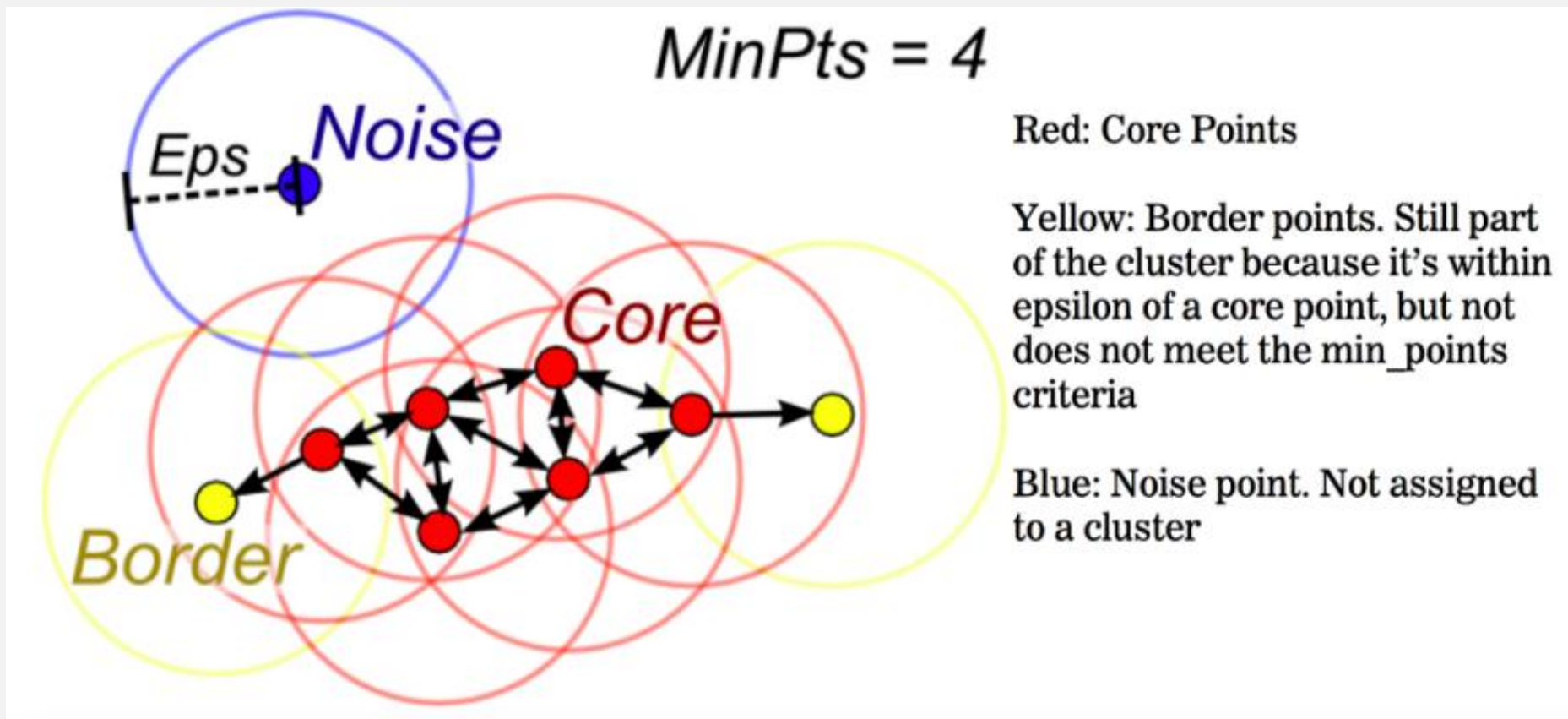
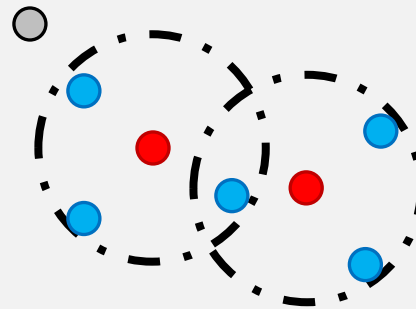


Image from: [link](#)

DBSCAN Concepts – Defining a cluster

- So, we saw in DBSCAN, we have **core points**, **border points**, and **outlier points**.
- How do we define a cluster?
- Outliers are not assigned a cluster in DBSCAN, they are treated as noise.
- A cluster is a set of core points and border points such that:
 - Neighboring core points belong to the same cluster.
 - Border point neighboring a core point belongs to the same cluster as the core point.
 - Two core points that share a common border point belong to the same cluster.



minPts = 4 (notice the point itself is counted)
Red are core points
Blue are border points
Grey are outliers

Two core points sharing a common border point belong to same cluster. Also notice the outlier is in neighborhood of a border point, but is still an outlier because it is not a core nor a border point.

Now that we have covered the concepts of DBSCAN. It is useful to see the animation video again [link](#)

```
class sklearn.cluster.DBSCAN(eps=0.5, *, min_samples=5,  
metric='euclidean', metric_params=None, algorithm='auto', leaf_size=30,  
p=None, n_jobs=None) \[source\]
```

Parameters:

eps : float, default=0.5

The maximum distance between two samples for one to be considered as in the neighborhood of the other. This is not a maximum bound on the distances of points within a cluster. This is the most important DBSCAN parameter to choose appropriately for your data set and distance function.

min_samples : int, default=5

The number of samples (or total weight) in a neighborhood for a point to be considered as a core point. This includes the point itself.

```
1 from sklearn.cluster import DBSCAN  
2  
3 dbscan_clustering = DBSCAN(eps=3, min_samples=2)  
4 dbscan_clustering.fit(X)  
5 dbscan_clustering.labels_ # returns cluster assignment for X
```

DBSCAN Strengths and Weaknesses

Strengths:

- No need to define number of clusters.
- Finds arbitrary shaped clusters based on density.
- Not all points are clustered; outliers are treated as noise by DBSCAN. Thus, DBSCAN has additional use-case of detecting outliers.
- Scales well. Average runtime is $O(m \log m)$.
- Any distance metric can be used.

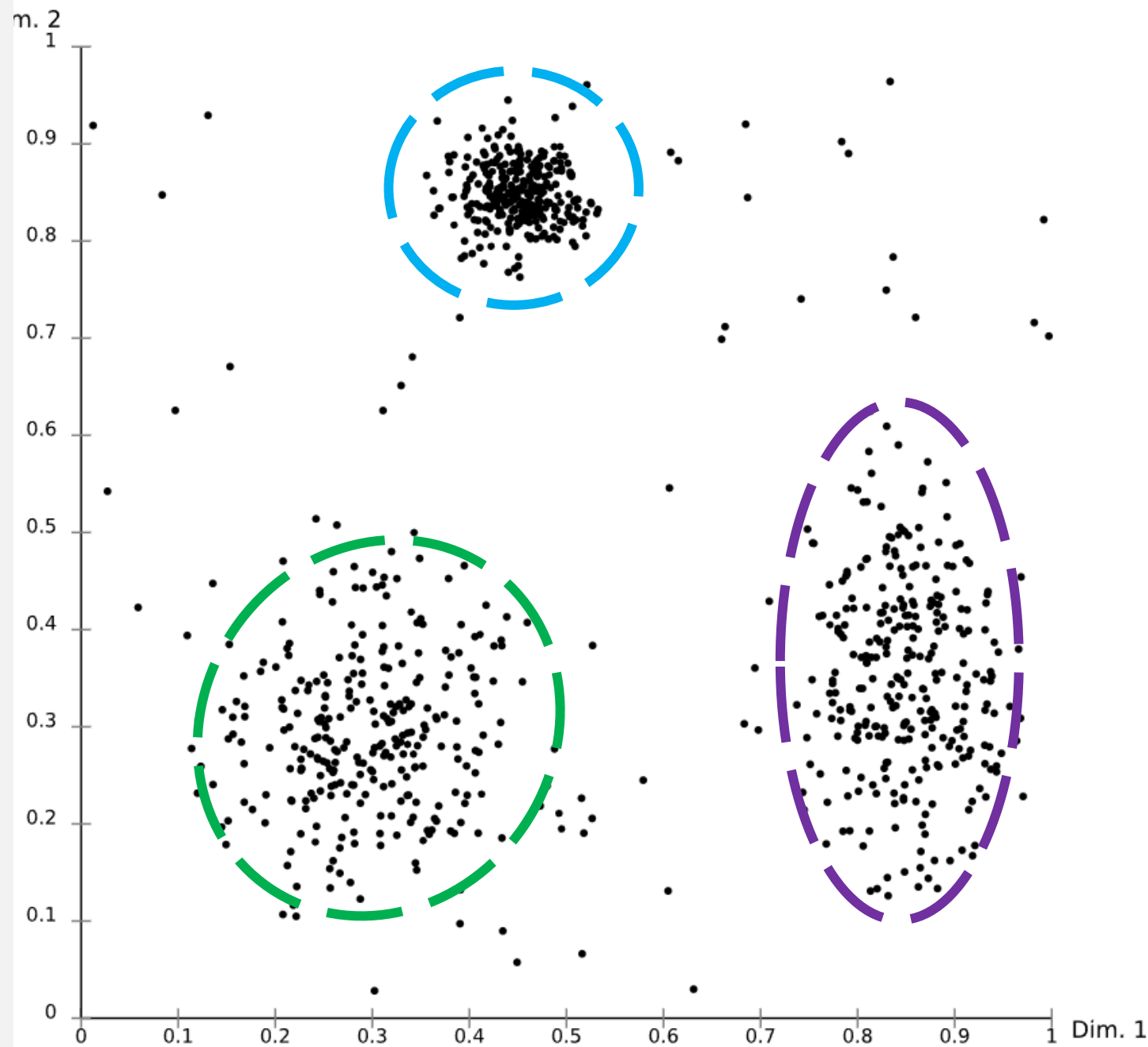
Weaknesses:

- Datasets that have varying **densities** are not clustered well by DBSCAN. This is because we set a single **eps** and **minPts** which governs the density for ALL clusters. OPTICS is a variation of DBSCAN that remedies this.
- Choosing the right parameter settings can be difficult.

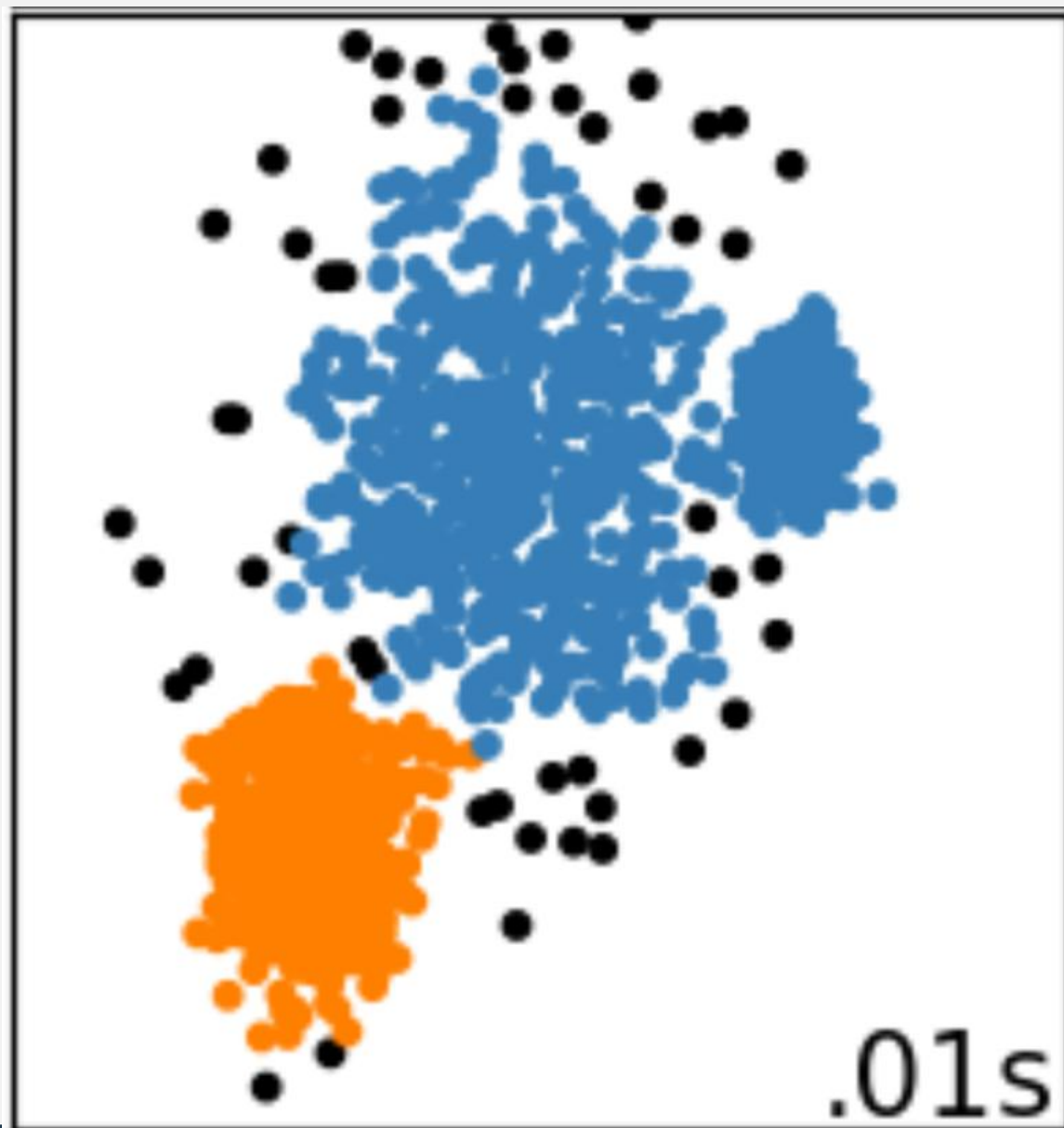
DBSCAN Weakness: dataset of varying density

- Datasets that have varying **densities** are not clustered well by DBSCAN.
- This is because we set a single ϵ and minPts which governs the density for ALL clusters.
- For the dataset on the right, we can see three natural clusters.
- But they have varying density, and as a result varying ϵ and minPts to capture them.
- OPTICS is a variation of DBSCAN that remedies this. See [link](#) for those interested.

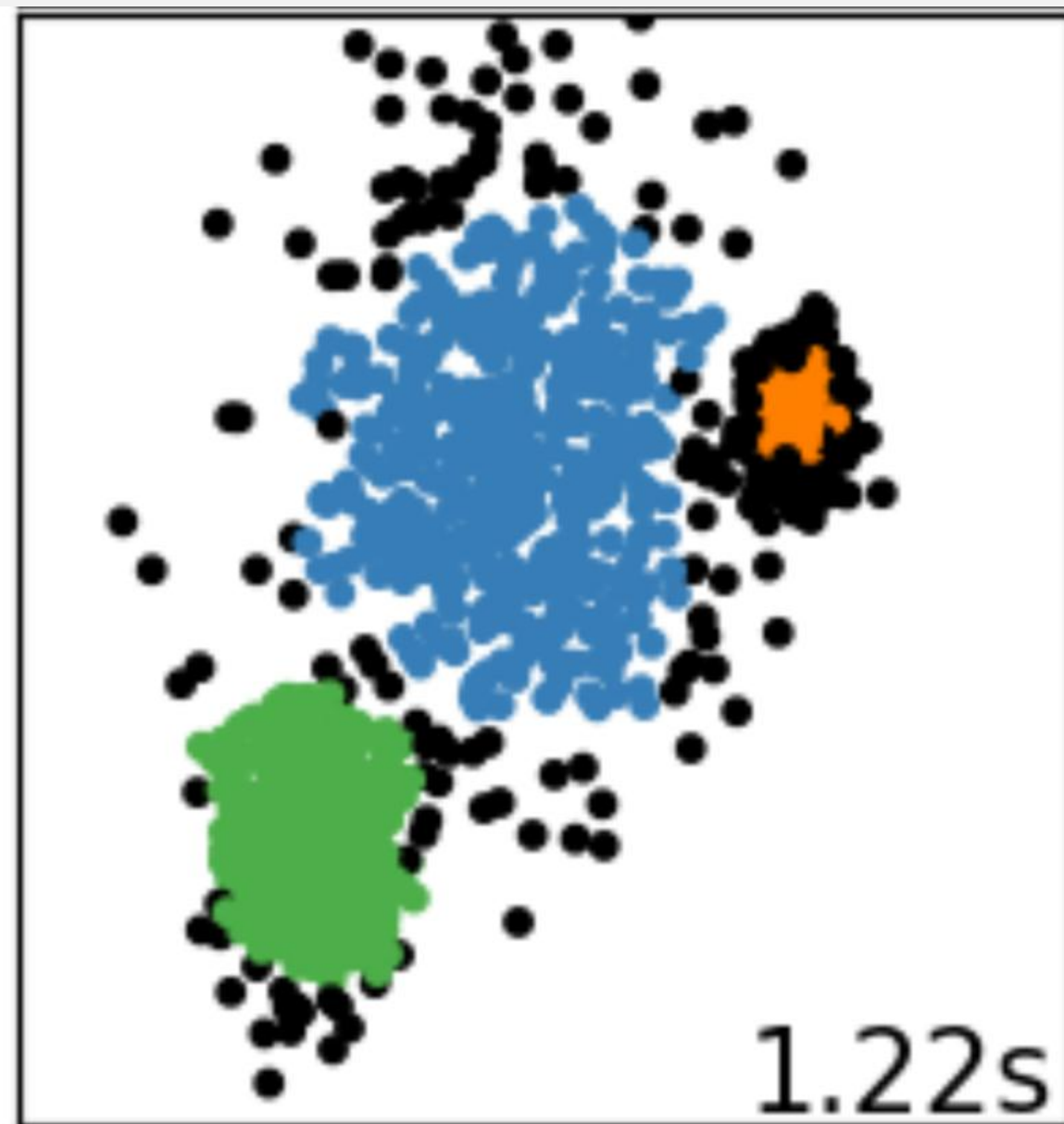
Image from: [link](#)



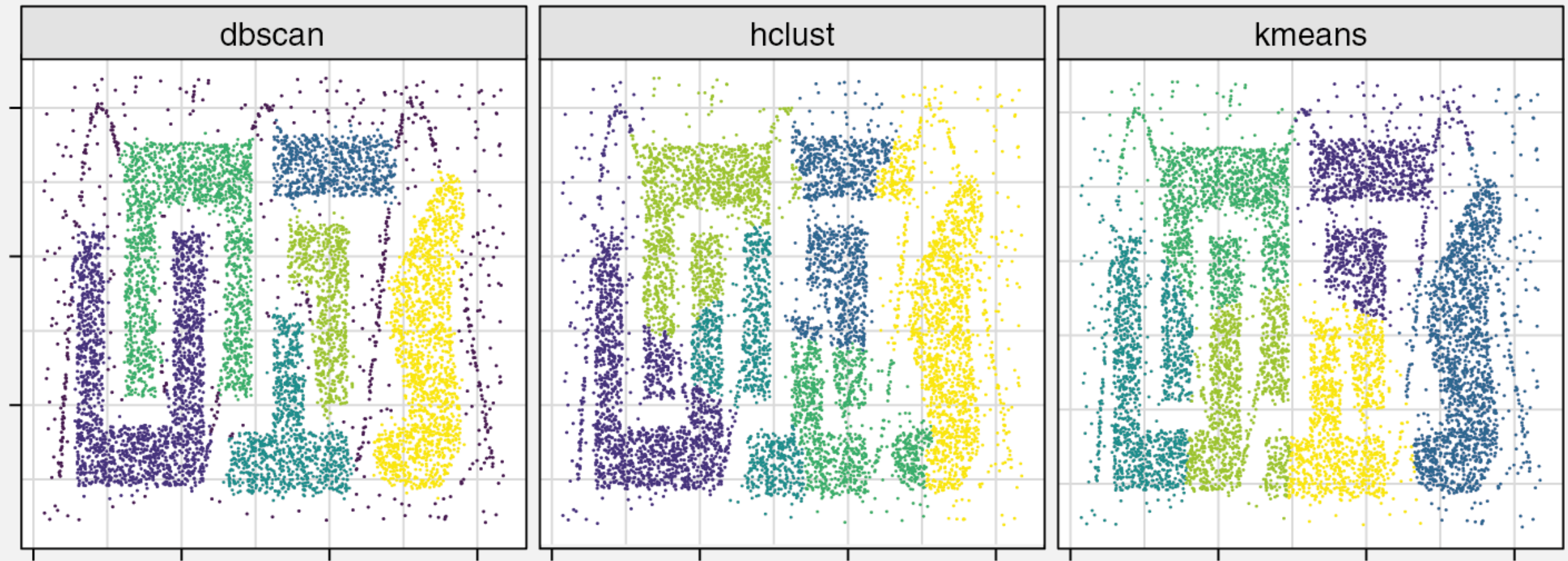
DBSCAN



OPTICS



DBSCAN vs HAC vs kmeans example [1]



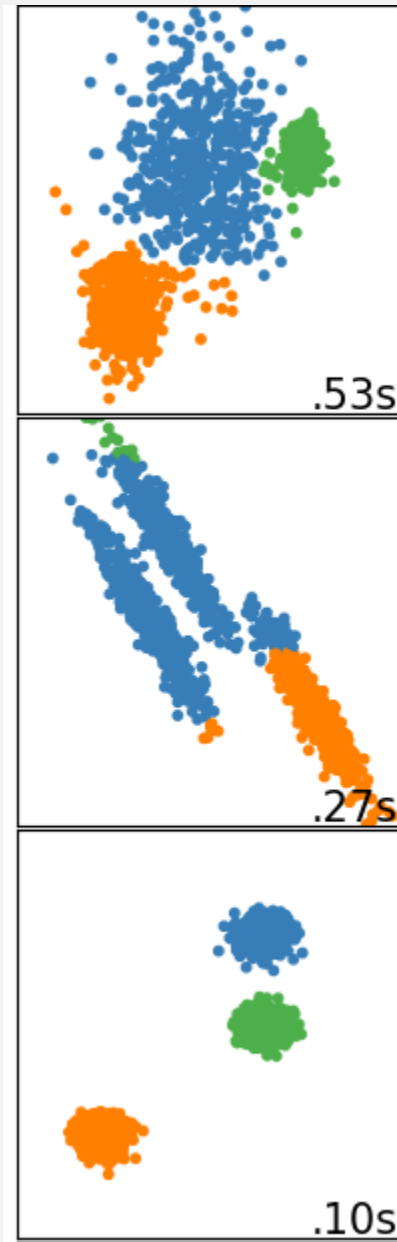
See more examples at: [link](#)

Image from: [link](#)

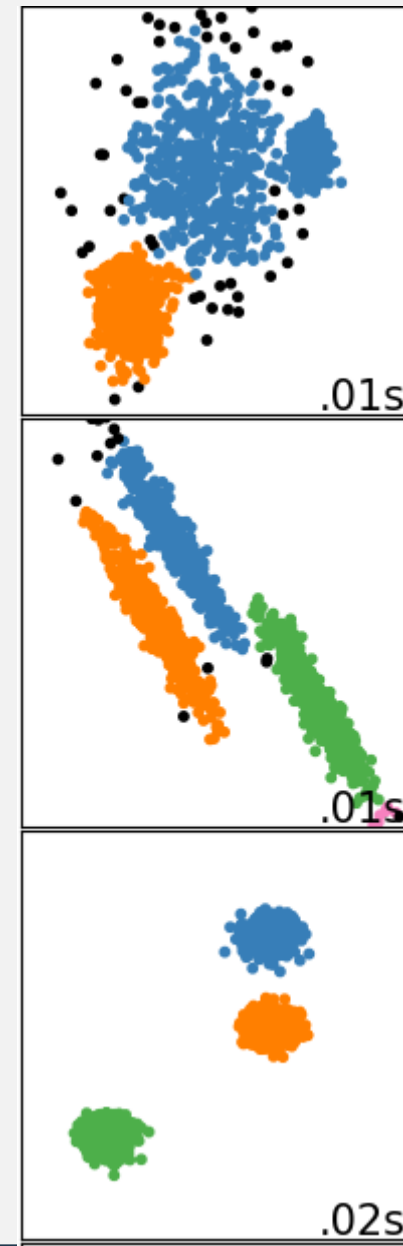
K-means



HAC



DBSCAN



See more examples at: [link](#)

	Method name	Parameters	Scalability	Usecase	Geometry (metric used)
★	K-Means	number of clusters	Very large <code>n_samples</code> , medium <code>n_clusters</code> with MiniBatch code	General-purpose, even cluster size, flat geometry, not too many clusters, inductive	Distances between points
	Affinity propagation	damping, sample preference	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Graph distance (e.g. nearest-neighbor graph)
	Mean-shift	bandwidth	Not scalable with <code>n_samples</code>	Many clusters, uneven cluster size, non-flat geometry, inductive	Distances between points
	Spectral clustering	number of clusters	Medium <code>n_samples</code> , small <code>n_clusters</code>	Few clusters, even cluster size, non-flat geometry, transductive	Graph distance (e.g. nearest-neighbor graph)
	Ward hierarchical clustering	number of clusters or distance threshold	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, transductive	Distances between points
★	Agglomerative clustering	number of clusters or distance threshold, linkage type, distance	Large <code>n_samples</code> and <code>n_clusters</code>	Many clusters, possibly connectivity constraints, non Euclidean distances, transductive	Any pairwise distance
★	DBSCAN	neighborhood size	Very large <code>n_samples</code> , medium <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, outlier removal, transductive	Distances between nearest points
	OPTICS	minimum cluster membership	Very large <code>n_samples</code> , large <code>n_clusters</code>	Non-flat geometry, uneven cluster sizes, variable cluster density, outlier removal, transductive	Distances between points
	Gaussian mixtures	many	Not scalable	Flat geometry, good for density estimation, inductive	Mahalanobis distances to centers
	BIRCH	branching factor, threshold, optional global clusterer.	Large <code>n_clusters</code> and <code>n_samples</code>	Large dataset, outlier removal, data reduction, inductive	Euclidean distance between points

	K-means	HAC	DBSCAN
Hyperparameters	number of clusters	number of clusters, linkage type, distance metric	eps, minPts
Use-case	General-purpose, even cluster sizes	Many clusters, non-Euclidean Distances,	uneven cluster sizes, outlier detection/removal
Type of clusters (shape)	Spherical shape	Depends on linkage. Single-link can give long and thin clusters. Complete-link can give spherical.	Density based clusters of arbitrary shape
Distance metric	Fixed to Euclidean distance.	any pairwise distance can be used	any pairwise distance can be used
Time complexity (scalability)	Scales well to large datasets. Runtime complexity is $O(KmdN)$	Does not scale well. Runtime is $O(m^3)$, optimized implementations are $O(m^2)$.	Scales well to large datasets. Average runtime is $O(m \log m)$.